COSC264 · INTRODUCTION TO COMPUTER NETWORKS AND THE INTERNET · TERM 4

Module 4
Reliable Data Transfer: ARQ Protocols

cosc264.up.railway.app

RECALL · MODULE 3

A quick recap

- Datagrams can get **corrupted** as they travel through the network.
- **Error detection codes** — parity check, Internet checksum, CRC — let the receiver detect whether a packet has been corrupted.
- **Forward Error Correction (FEC)** goes further: using error-correcting codes, the receiver can not only detect errors but also **correct** them without asking for a retransmission.

THE BIGGER PICTURE

What error correction cannot solve

Error correction alone cannot handle every failure. Two cases in particular:

CASE 1 — CORRUPTED BEYOND REPAIR

The packet arrives corrupted, and the corruption is **beyond correction**, so the receiver discards it. The sender receives **no signal** — as far as it knows, the packet was delivered.

CASE 2 — NEVER ARRIVES

The packet is **lost in transit** and never arrives at all. There is nothing to correct and nothing to detect — and again, **the sender has no idea**.

Error correction alone is not enough. We need a **concrete protocol between the sender and the receiver** to coordinate and handle both cases.

MODULE 4 · CONCEPT 1 OF 4

Reliable Data Transfer over an Unreliable Channel

Reliable Data Transfer and Automatic Repeat reQuest

1 · Reliable data transfer | 2 · Stop-and-wait RDT | 3 · Go-back-N | 4 · Selective repeat

THIS MODULE

Reliable Data Transfer (RDT)

Reliable Data Transfer (RDT) is the goal: deliver data correctly, completely, and in order to the layer above (typically the application) — no matter what the underlying channel does to it.

For example, suppose the underlying channel is **IP**. IP is only **best-effort**: packets might be lost, might arrive corrupted, or might arrive out of order.

If we run an RDT protocol on top of IP, it compensates for all of those risks and still provides correct, complete, in-order delivery.

MODULE 4

Outline

We build up **reliable data transfer (RDT)** protocols one channel assumption at a time:

- **rdt 1.0** — over a perfectly reliable channel
- **rdt 2.0** — over a channel with **bit errors**
- **rdt 3.0** — over a **lossy** channel with bit errors

ARQ — Automatic Repeat reQuest

Most RDT protocols are built on a core idea: if the sender does not hear back, it **retransmits**. Protocols following this pattern are called **ARQ** (Automatic Repeat reQuest) protocols.

By the end of this module, we will learn three ARQ protocols, in increasing sophistication:

- **Stop-and-wait** ARQ — the basic version, the ARQ underlying **rdt 3.0**.
- Building on it, a **pipelined** and more efficient version: **go-back-N** ARQ.
- **Selective repeat** ARQ — pipelined, and more efficient still.

MODULE 4 · CONCEPT 2 OF 4

Stop-and-wait RDT

Building the protocol up from rdt 1.0 to rdt 3.0

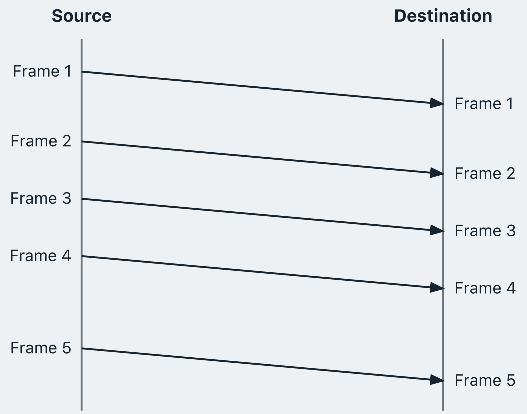
RDT 1.0

rdt 1.0 — a perfectly reliable channel

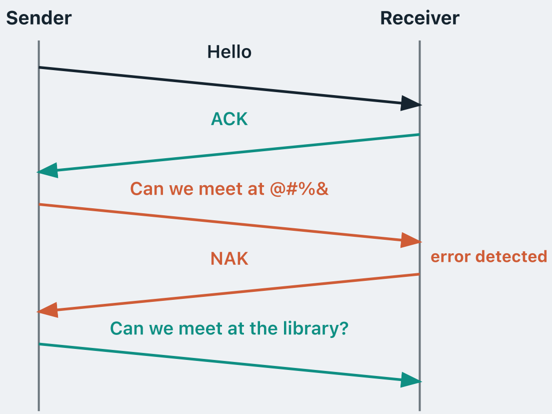
Start with the easiest case — a **perfectly reliable channel**:

- no **bit errors**
- no **out-of-order** frames
- no **frame loss**

> Nothing can go wrong, so there is nothing for the protocol to do: the sender just sends, and the receiver just delivers what arrives.



rdt 2.0 — a channel with only bit errors



Bits can now be flipped in transit.

Here's one idea: the receiver runs an **error detection code** — a checksum or CRC — exactly what we learned in Module 3.

If it arrives intact, the receiver sends a **positive acknowledgement (ACK)**.

If an error is spotted, the receiver sends a **negative acknowledgement (NAK)**.

On receiving a NAK, the sender **retransmits**.

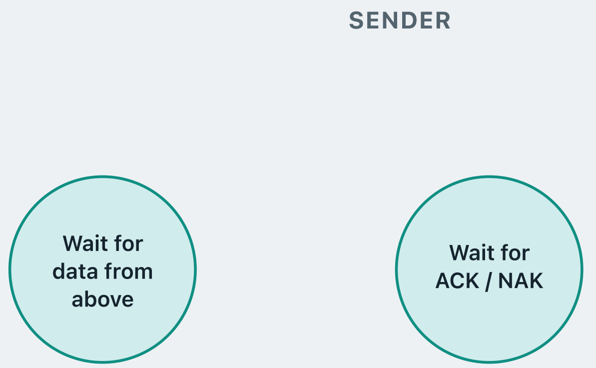
RDT 2.0

The protocol as a state machine

- We describe what the sender does and what the receiver does with one **finite-state machine** each.
- A finite-state machine sits in one of several **states**, with arrows for the **transitions** between them — and each arrow is labelled with the event that triggers that transition.

RDT 2.0

The protocol as a state machine



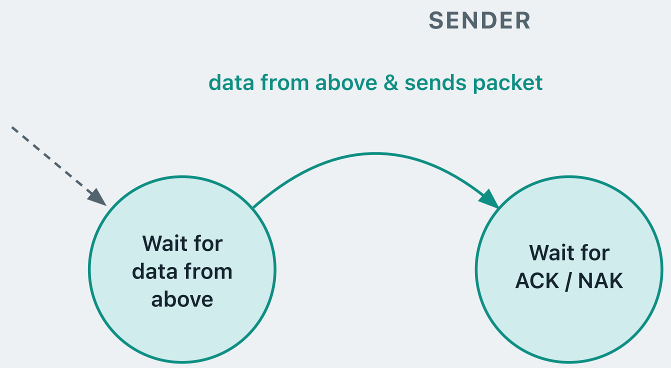
The sender's state machine has two states.

Wait for data from above — the sender is idle, waiting for the next packet from the application.

Wait for ACK / NAK — the sender has just sent a packet to the receiver, and has not heard back yet.

RDT 2.0

The protocol as a state machine

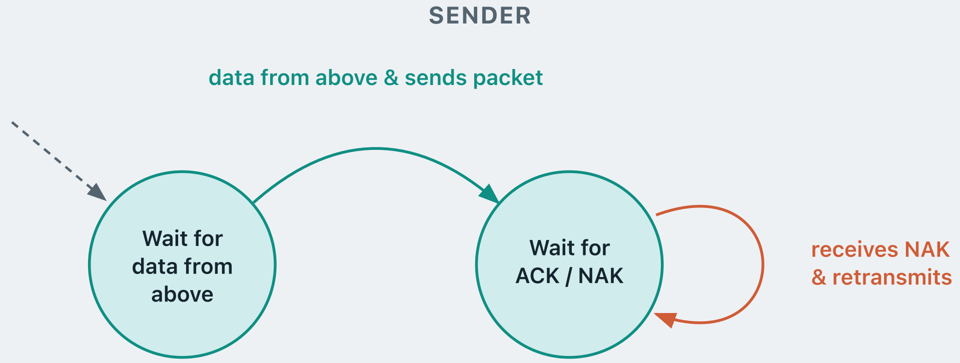


data from above & sends packet

The application hands data down. The sender builds the packet, transmits it, and moves to the waiting state.

RDT 2.0

The protocol as a state machine



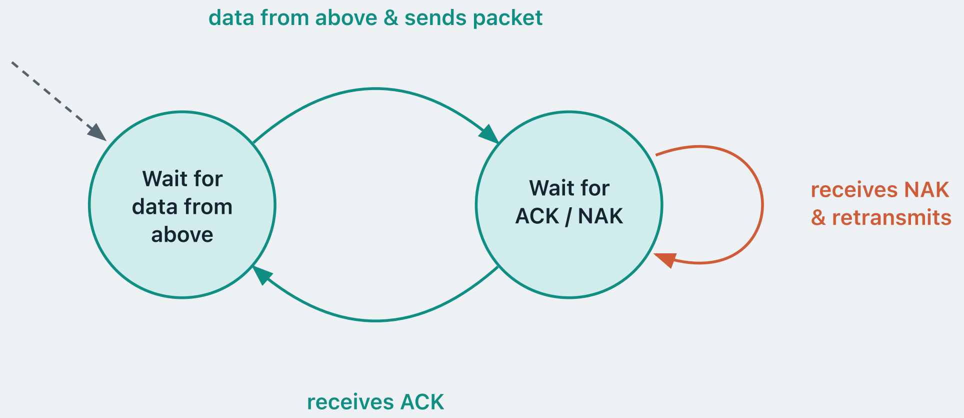
receives NAK & retransmits

The receiver reported a corrupted packet, so the sender retransmits **the same packet** — and comes straight back to the same waiting state.

RDT 2.0

The protocol as a state machine

SENDER

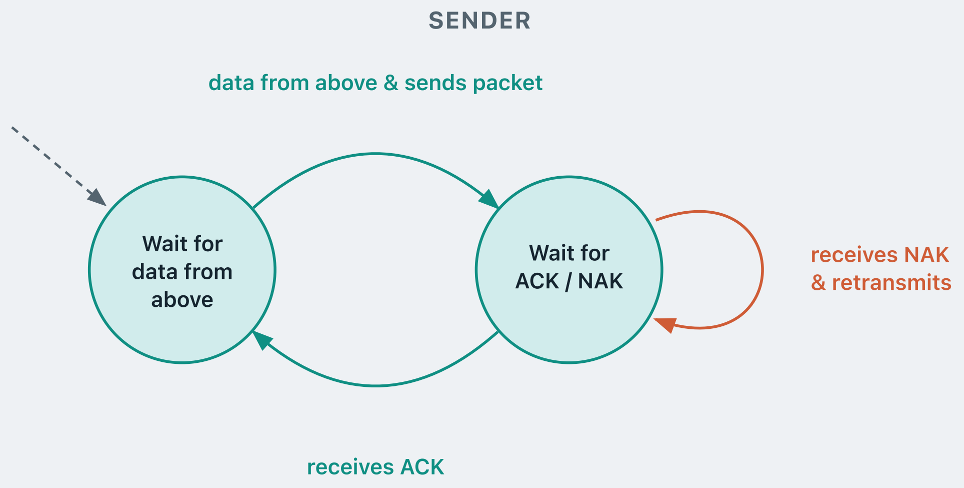


receives ACK

The packet got through. The sender returns to the idle state, ready for the next packet from above.

RDT 2.0

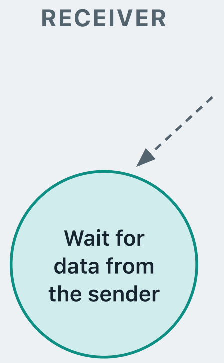
The protocol as a state machine



This is the **stop-and-wait** strategy: the sender transmits **one single packet**, then **stops** and does nothing at all until it hears back from the receiver.

RDT 2.0

The protocol as a state machine

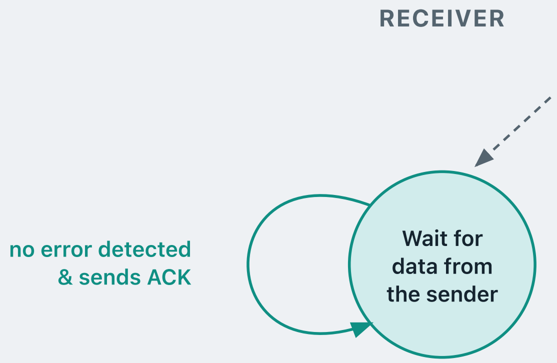


The receiver's state machine has one state.

Wait for data from the sender — it just waits for the next packet to arrive from the channel.

RDT 2.0

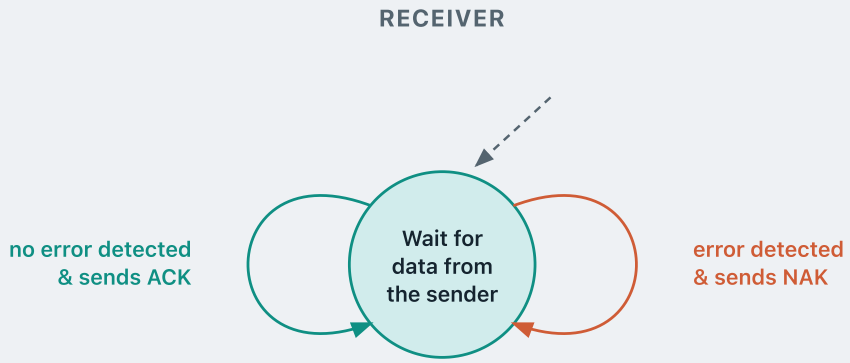
The protocol as a state machine



no error detected & sends ACK

Deliver the data to its application, and send an ACK back to the sender.

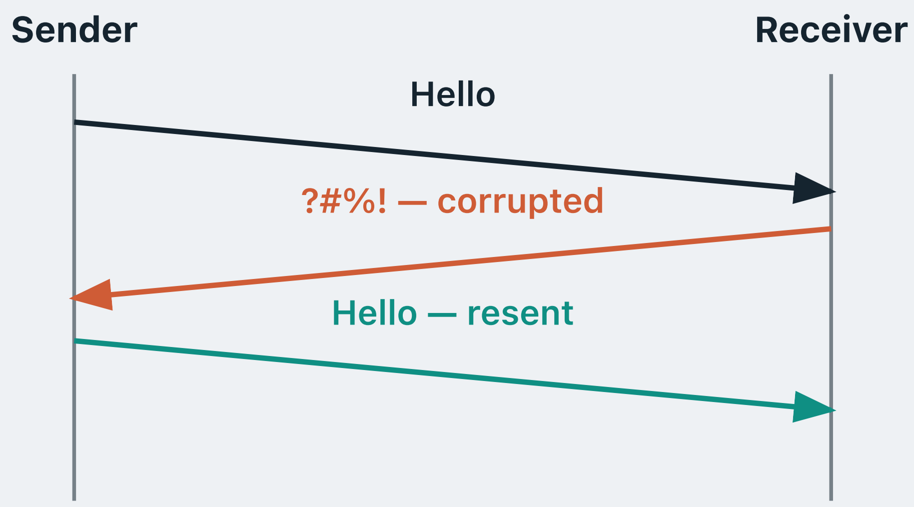
The protocol as a state machine



error detected & sends NAK

Discard the packet, and send a NAK.

A fatal flaw



We assume that the **data** sent from the sender can be corrupted. But the **ACK and NAK** messages sent by the receiver also travel back through the **same channel** — so they can also be corrupted.

Suppose the sender sends "Hello". The receiver gets it intact and sends back an **ACK**.

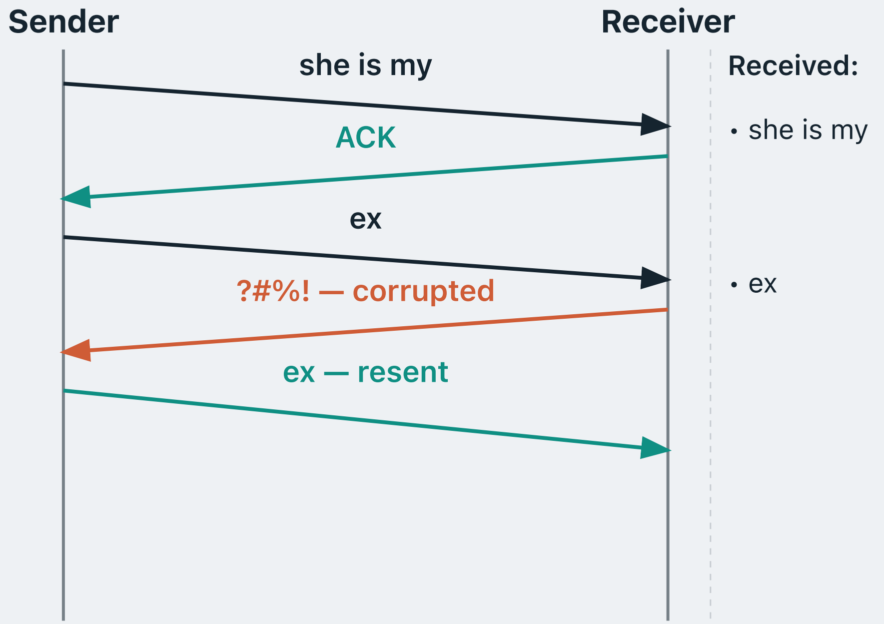
But the ACK gets **corrupted** in transit.

What should the sender do?

The sender cannot tell whether the receiver said ACK or NAK. Its only safe option is to **resend**.

RDT 2.0

A fatal flaw

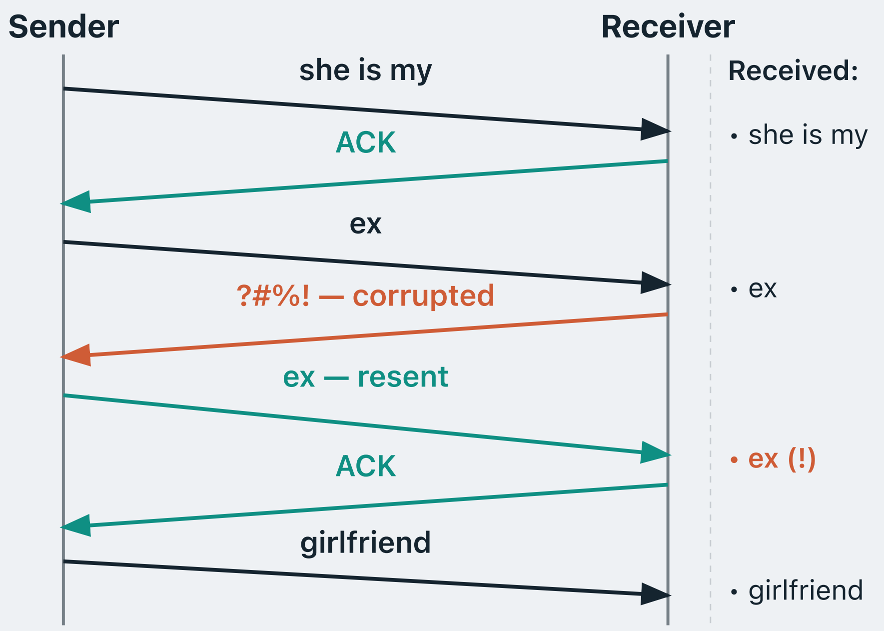


Suppose the sender wants to deliver the message **“she is my ex girlfriend”** from the application above. **“she is my”** arrives intact — receiver delivers it and sends ACK.

“ex” arrives intact — receiver delivers it and sends ACK.

The ACK gets **corrupted** — sender cannot tell, so it **resends** “ex”.

A fatal flaw



Receiver has no idea this is a resend — it **delivers “ex” again**.

“girlfriend” eventually gets through and is delivered.

> In the end, the receiver delivers “she is my **ex ex** girlfriend”.
>
> This is the wrong message — it contains a **duplicate “ex”**.

How do we fix this?

RDT 2.1

The fix: a 1-bit sequence number

The fix is as simple as tagging every packet with a **sequence number**, so the receiver can tell **new data** from a **resend**.

For stop-and-wait, a **single bit** is enough — the sender alternates **0** and **1**.

SENDER

- **New data:** flip the sequence bit and transmit.
- **Retransmission:** resend with the **same** sequence number.

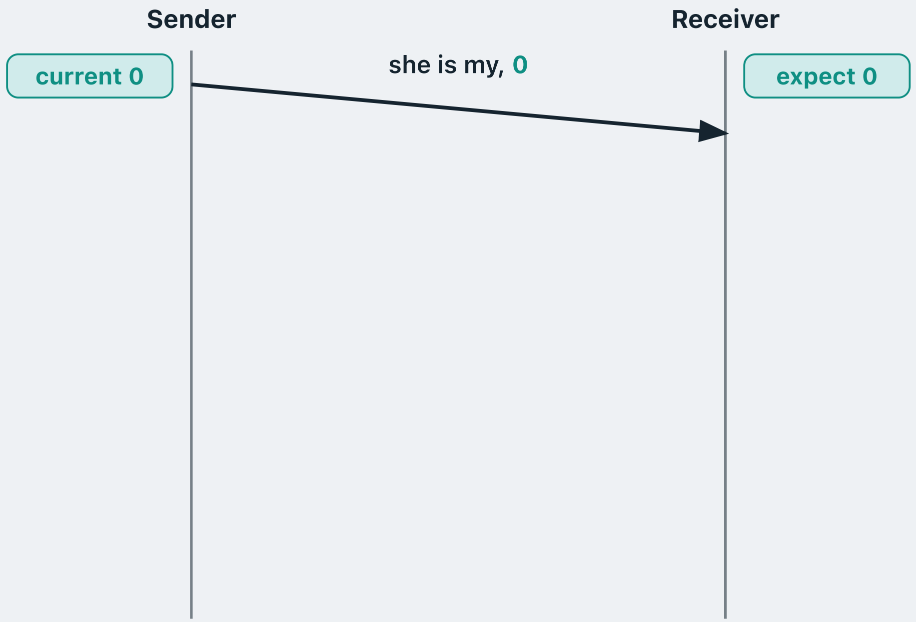
RECEIVER

- Packet with the **same** number as last time → **duplicate**; discard it, but re-send the ACK.
- Packet with a **different** number → **new data**; deliver it and ACK.

This protocol is known as **rdt 2.1**.

RDT 2.1

Catching a real duplicate



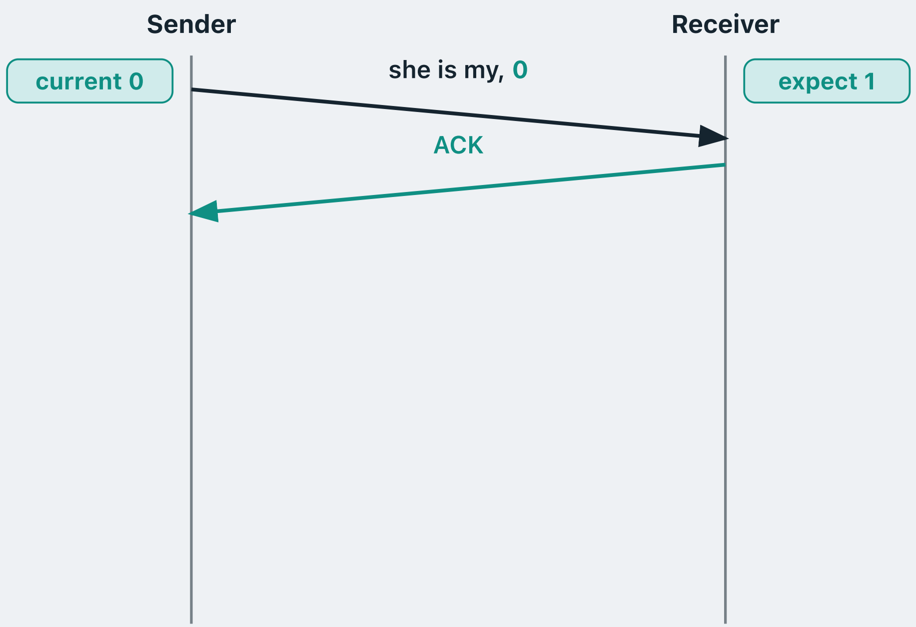
Same story as before: the sender wants to deliver **“she is my ex girlfriend”**.

Both sides start from **0**. The sender tags **“she is my”** with **0** and sends it.

0 is exactly the number the receiver is expecting, so this is **new data**.

RDT 2.1

Catching a real duplicate



Same story as before: the sender wants to deliver **“she is my ex girlfriend”**.

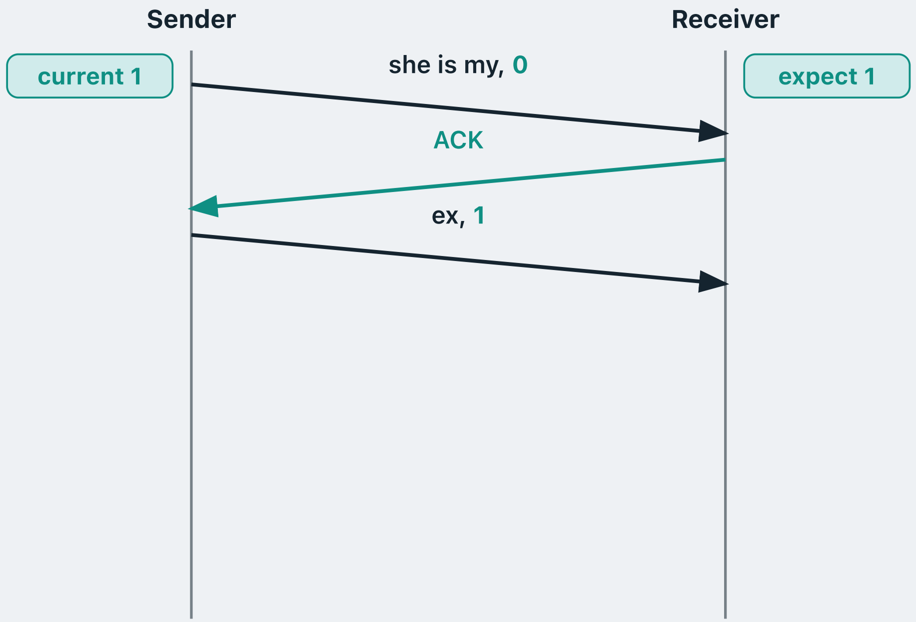
Both sides start from **0**. The sender tags **“she is my”** with **0** and sends it.

0 is exactly the number the receiver is expecting, so this is **new data**.

So the receiver **delivers the chunk**, sends an **ACK**, and now **expects 1**.

RDT 2.1

Catching a real duplicate



Same story as before: the sender wants to deliver **“she is my ex girlfriend”**.

Both sides start from **0**. The sender tags **“she is my”** with **0** and sends it.

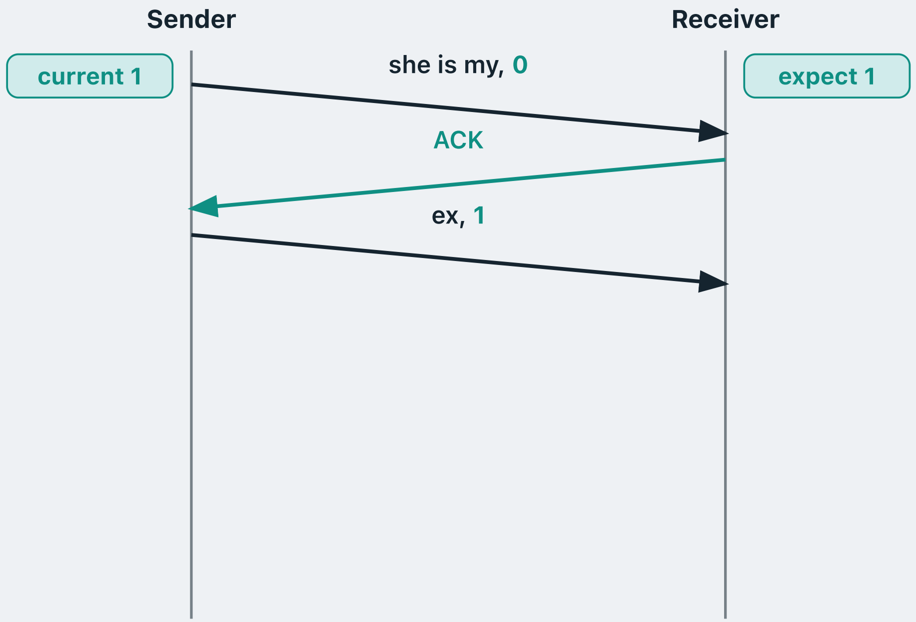
0 is exactly the number the receiver is expecting, so this is **new data**.

So the receiver **delivers the chunk**, sends an **ACK**, and now **expects 1**.

“ex” is new data too, so the sender **flips its bit** to **1** and sends it.

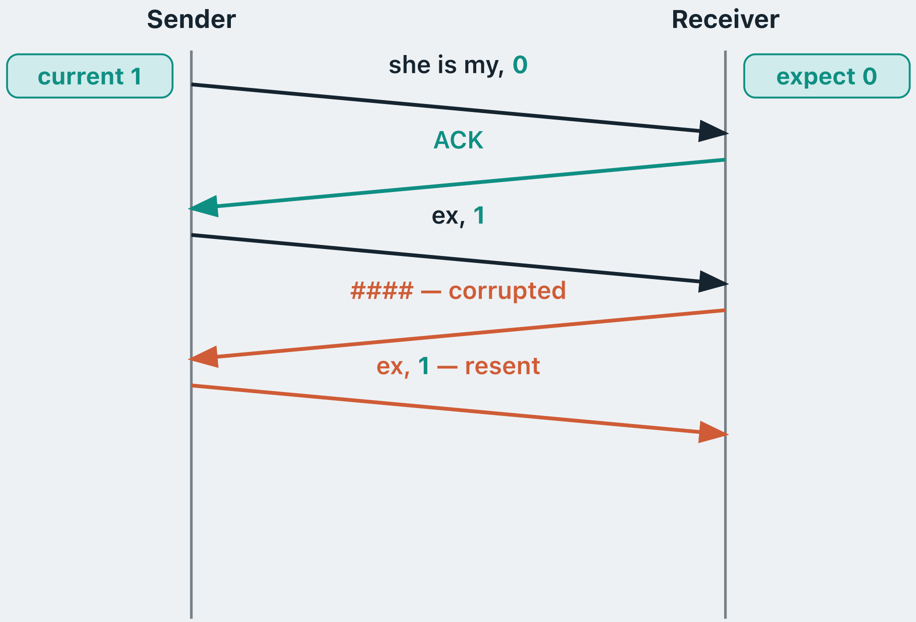
RDT 2.1

Catching a real duplicate



The receiver is **expecting 1** and this packet carries **1** — so this is **new data**.

Catching a real duplicate



The receiver is **expecting 1** and this packet carries **1** — so this is **new data**.

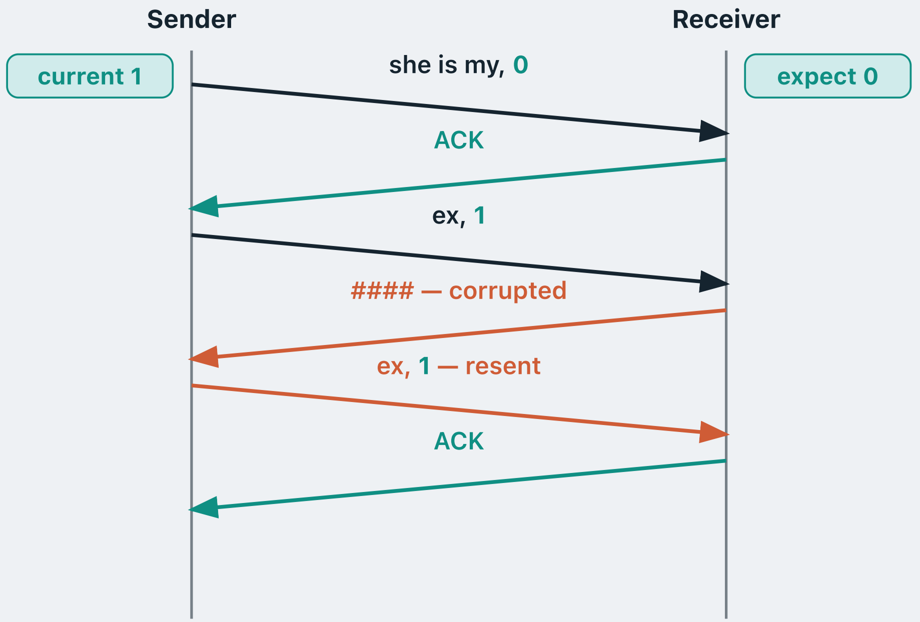
So the receiver **delivers the chunk**, sends an **ACK**, and now **expects 0**.

But this time that **ACK comes back corrupted**.

The sender cannot tell what the receiver said, so it **must resend** — this is a **retransmission**.

A retransmission is not new data, so the sender's current bit **stays at 1**: it sends **"ex, 1"** again.

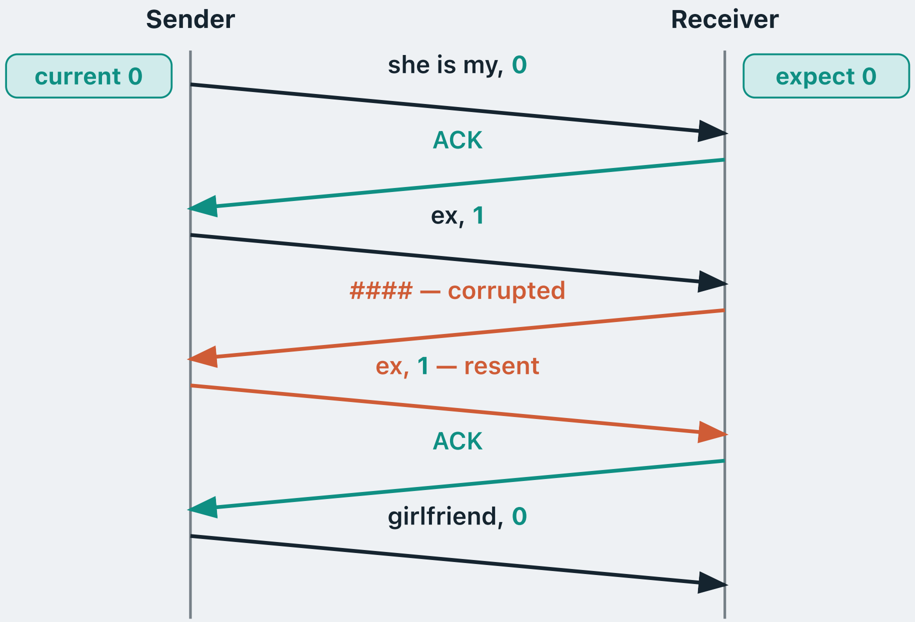
Catching a real duplicate



The receiver is **expecting 0**, but this packet carries **1** — so it is a **copy it already has**.

It **discards the copy** and just re-acknowledges, keeping its expected number at **0**.

Catching a real duplicate



The receiver is **expecting 0**, but this packet carries **1** — so it is a **copy it already has**.

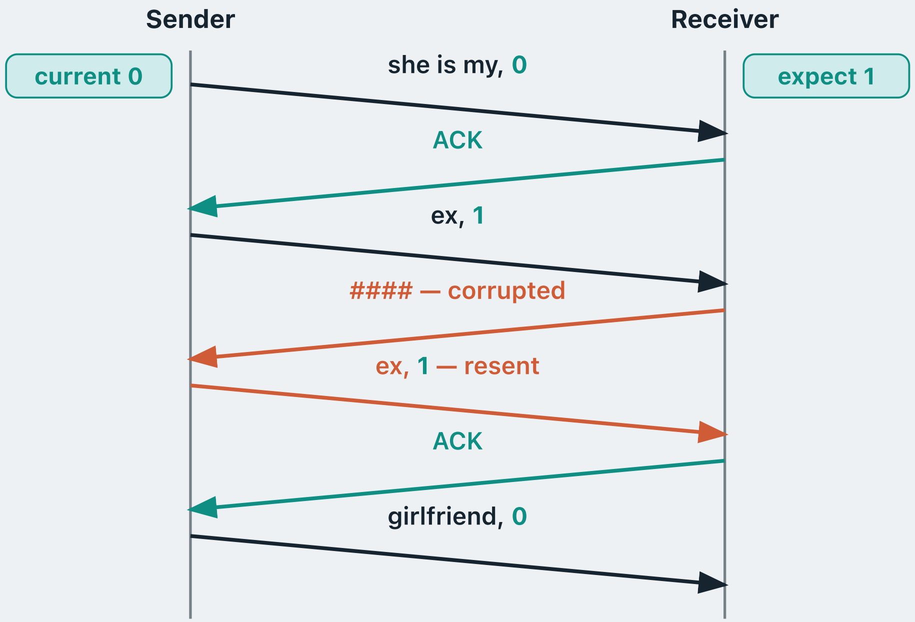
It **discards the copy** and just re-acknowledges, keeping its expected number at **0**.

The sender sees that ACK, so it can advance: it **flips to 0** and sends **"girlfriend"** with **0**.

0 is what the receiver expects, so this is **new data**.

RDT 2.1

Catching a real duplicate



The receiver is **expecting 0**, but this packet carries **1** — so it is a **copy it already has**.

It **discards the copy** and just re-acknowledges, keeping its expected number at **0**.

The sender sees that ACK, so it can advance: it **flips to 0** and sends **"girlfriend"** with **0**.

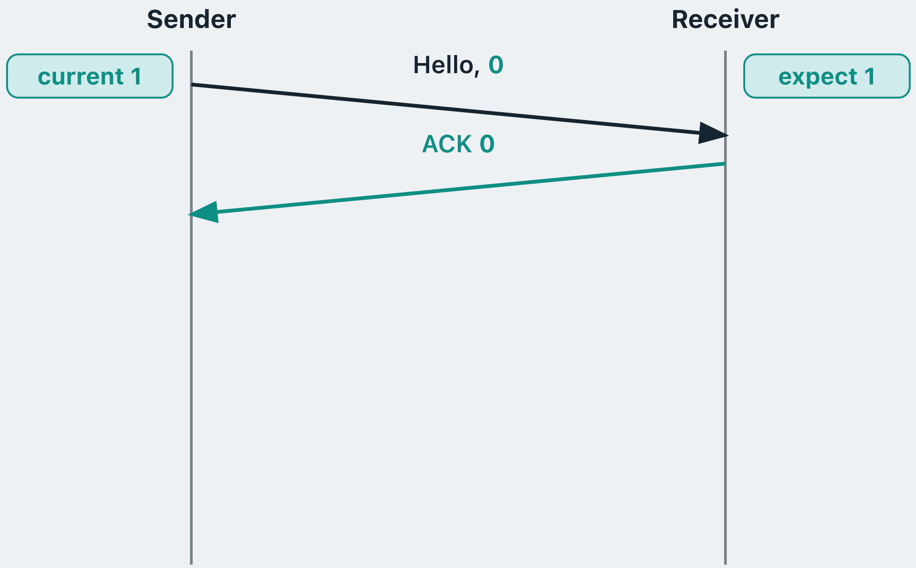
0 is what the receiver expects, so this is **new data**.

The receiver **delivers the chunk** and now **expects 1**.

"she is my ex girlfriend" ✓

Delivered correctly, with **no duplicate**.

Numbering the ACKs too

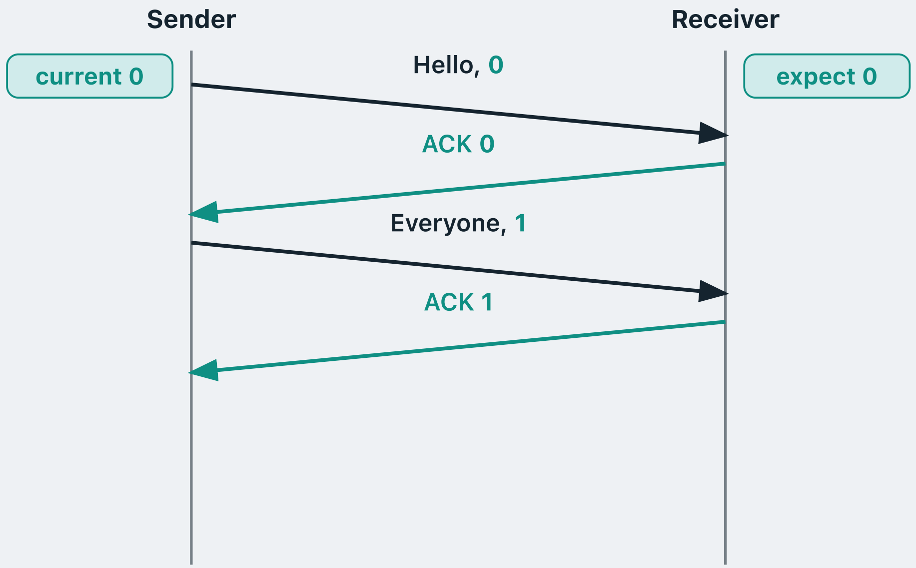


rdt 2.2 adds a neat trick: number the **receiver's** messages too.

Every ACK now carries a sequence number. **ACK 0** means "I received your packet **0**".

RDT 2.2

Numbering the ACKs too

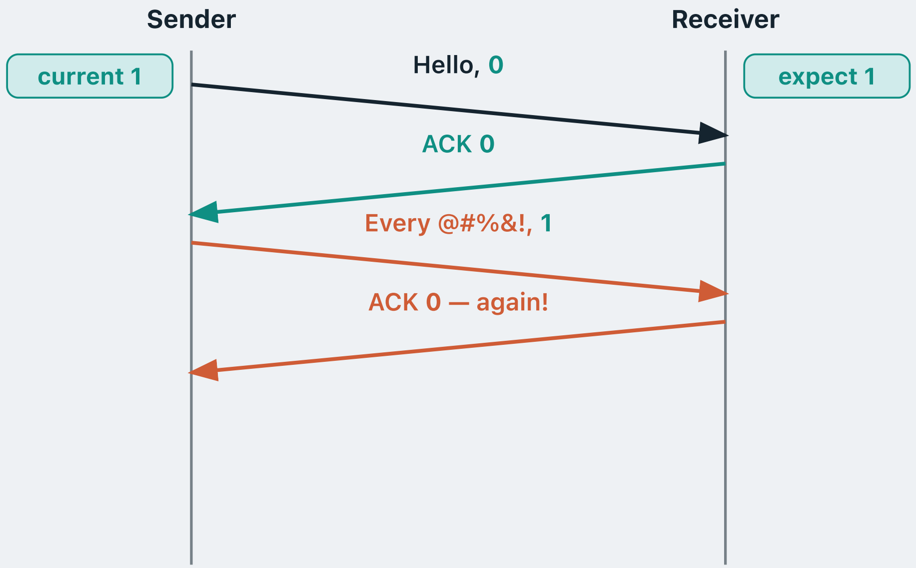


rdt 2.2 adds a neat trick: number the **receiver’s** messages too.

Every ACK now carries a sequence number. **ACK 0** means “I received your packet **0**”.

With nothing corrupted, the exchange simply alternates.

What if a packet is corrupted?



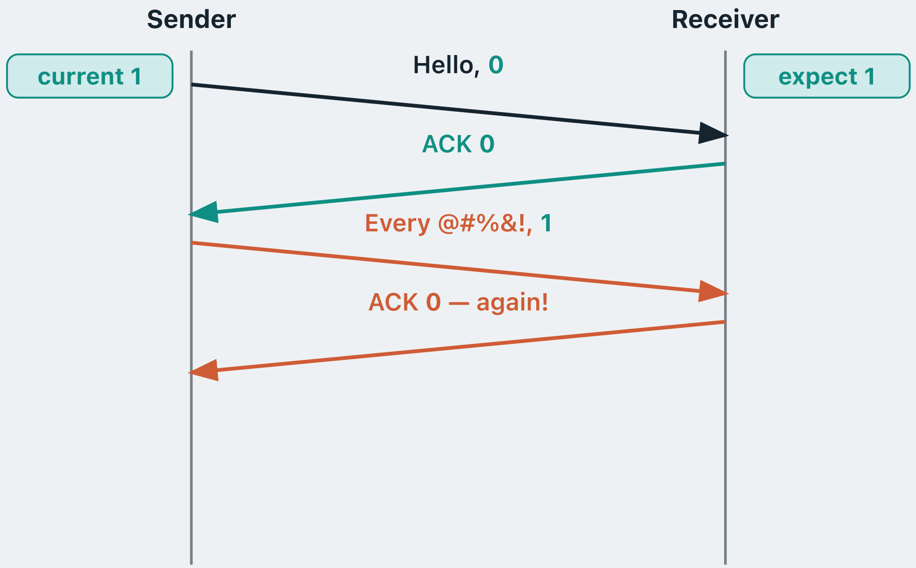
Now suppose that second packet arrives **corrupted**.

Normally the receiver would send a **NAK** — "I detected a corrupted packet, please send it again".

But its messages are numbered now. So instead of a NAK it just sends an ACK carrying the **previous** sequence number: **ACK 0**, again.

> Which says exactly this: "I still only have packet **0**. I have not received packet **1**."

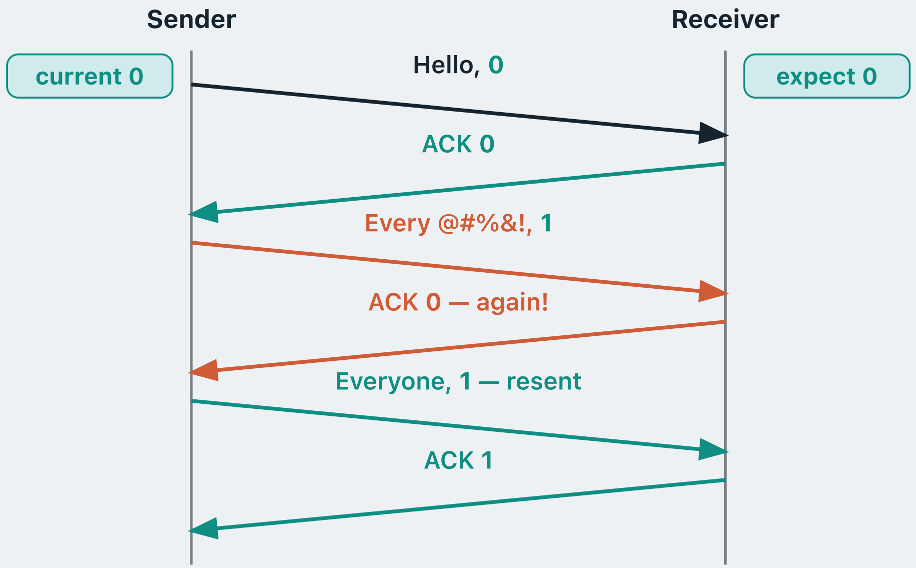
A duplicate ACK *is* a NAK



The sender is waiting for **ACK 1**. Getting **ACK 0** again tells it packet **1** never arrived — exactly what a NAK would have said.

> Seeing the same ACK number twice is called a **duplicate ACK**, and it does a NAK's job: it asks the sender to retransmit.

A duplicate ACK *is* a NAK



The sender is waiting for **ACK 1**. Getting **ACK 0** again tells it packet **1** never arrived — exactly what a NAK would have said.

> Seeing the same ACK number twice is called a **duplicate ACK**, and it does a NAK's job: it asks the sender to retransmit.

So the sender **retransmits**, using the **same sequence number** as before — **1** here.

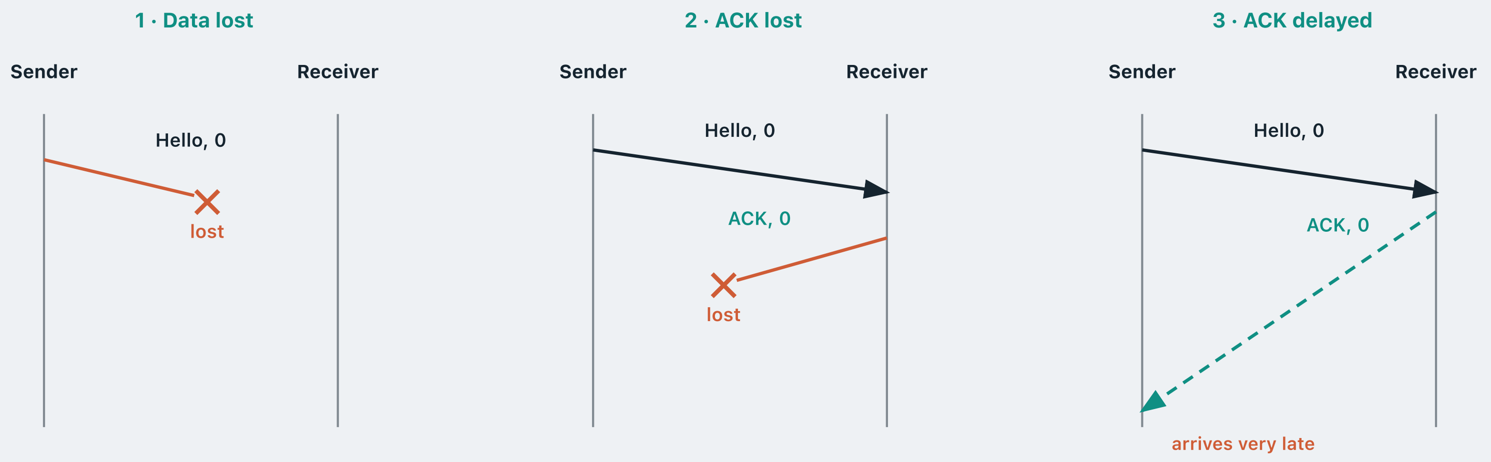
RDT 3.0

Three new ways to fail

In **rdt 3.0** we assume the channel is even more unstable.

It might not only corrupt bits, but can also **drop** a packet completely, or **delay** any packet significantly.

More specifically, three things can happen:



RDT 3.0

Timeout and retransmission

How do we resolve these problems?

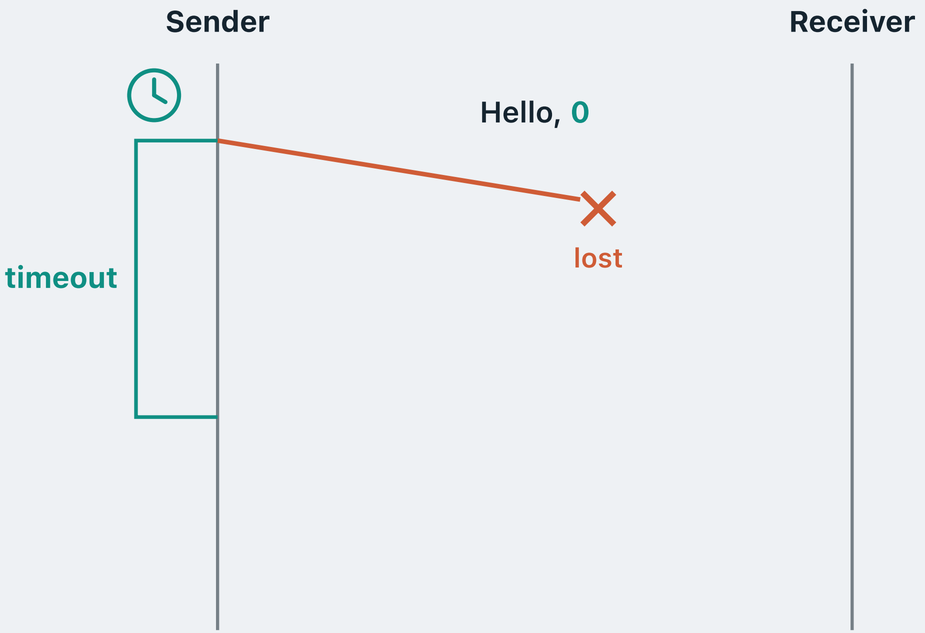
RDT 3.0

Timeout and retransmission

Surprisingly simply. The sender keeps a **timer**.

RDT 3.0

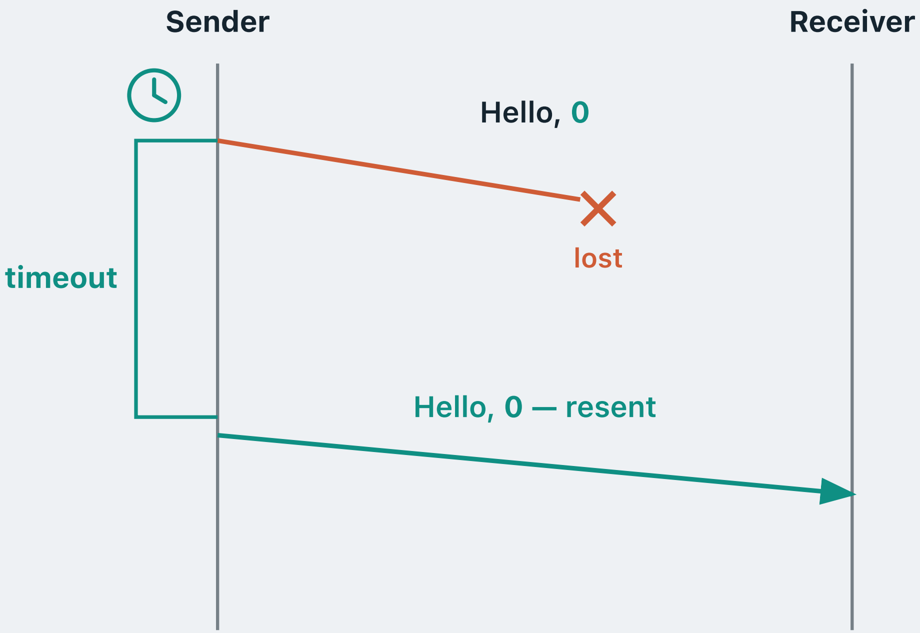
Timeout and retransmission



After a while the timer **expires**.

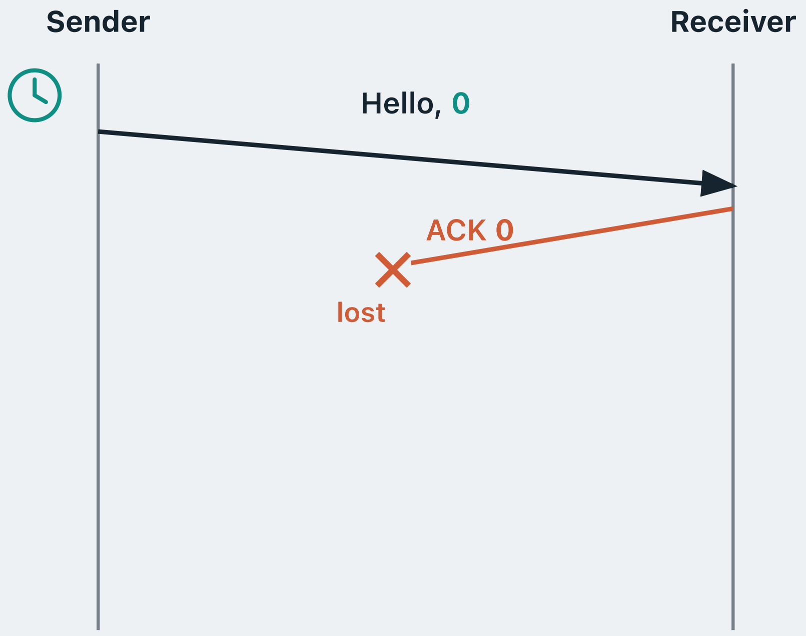
RDT 3.0

Timeout and retransmission



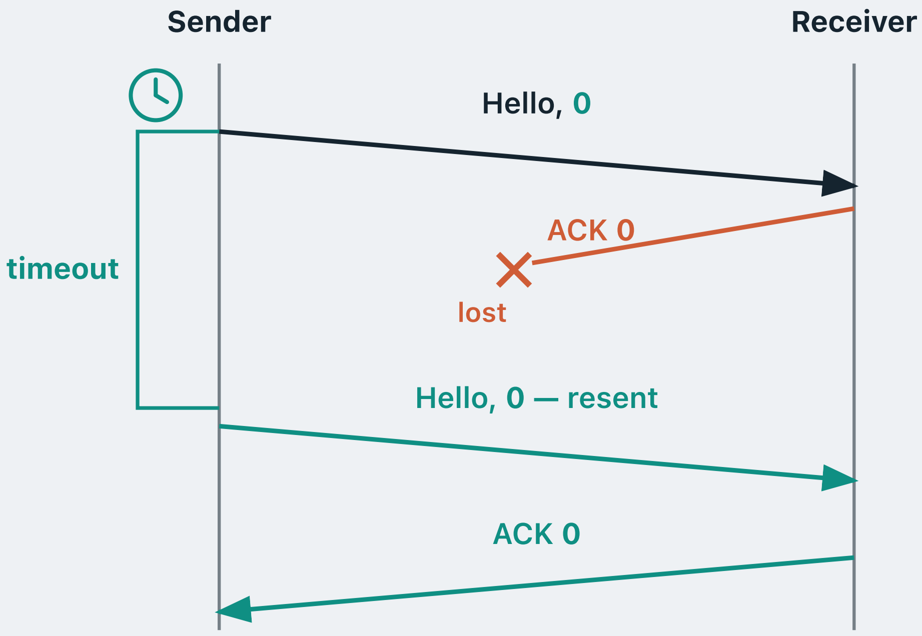
The sender assumes the worst — the packet is gone — and **retransmits it**, with the **same sequence number**.

Timeout and retransmission



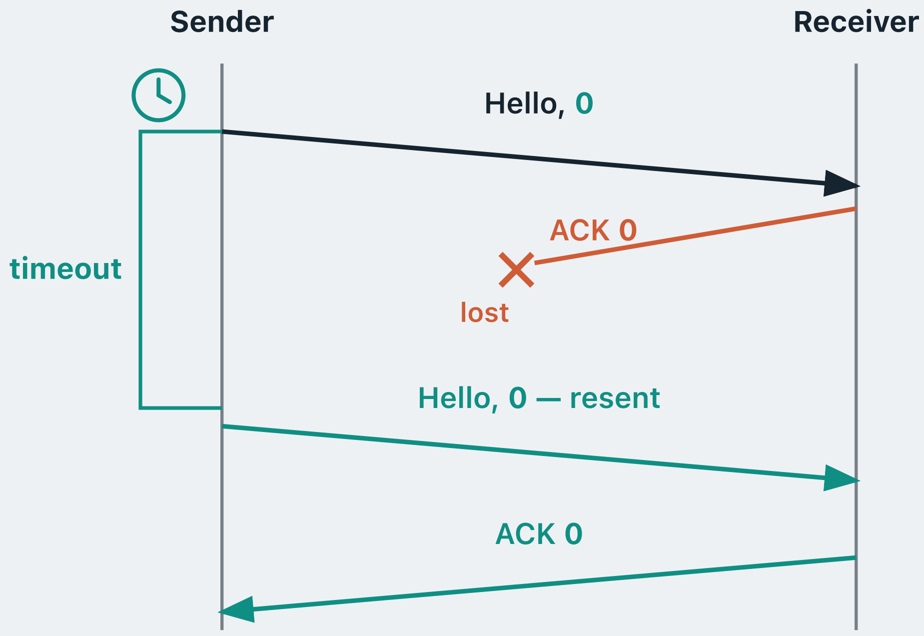
Now a different case: the data **does** arrive, but the **ACK is lost** on the way back.

Timeout and retransmission



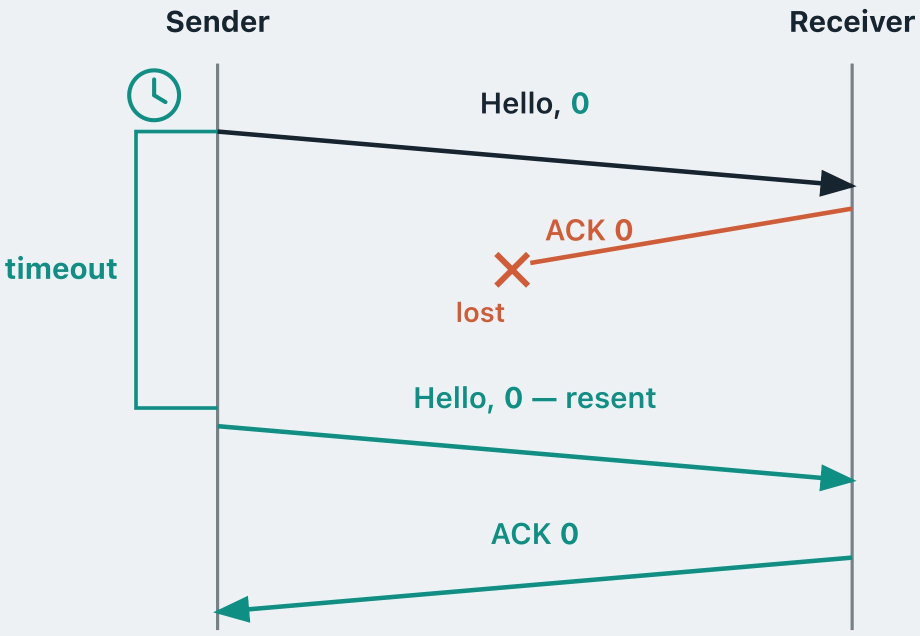
After the timer expires, the sender **still assumes the worst** — that the data was lost — and resends the **same packet** again.

Timeout and retransmission



The same happens if the ACK is merely **delayed** rather than lost: as long as the timer expires with no ACK heard, the sender always assumes the worst and **retransmits**.

Timeout and retransmission



But **how long should the sender wait?**

Too short

The ACK was merely slow, so the sender resends for nothing — **duplicate packets**.

Too long

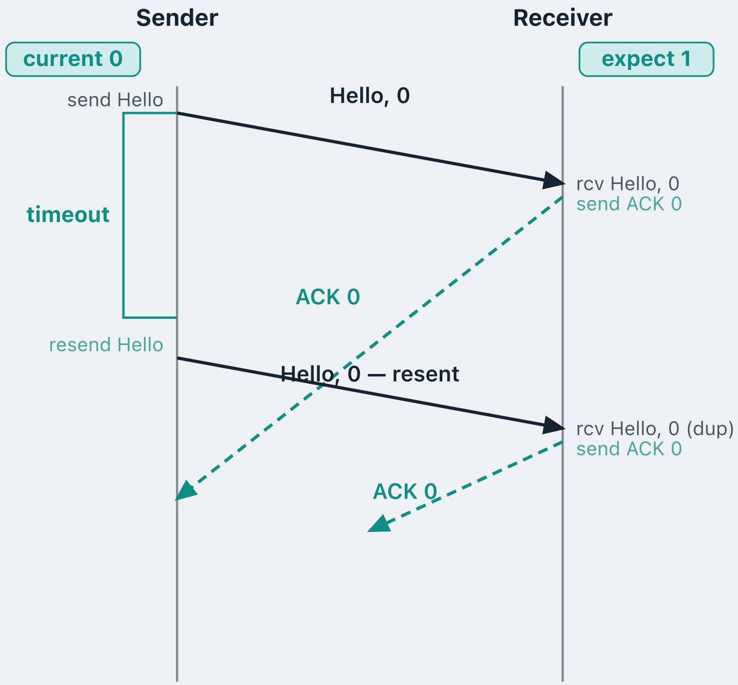
The sender sits idle long after the packet was lost — **wasted throughput**.

About **one RTT** — the time for a packet to reach the receiver and its ACK to come back.

We will come back to how that interval is chosen when we look at TCP.

A worked example

The sender wants to deliver two messages to the receiver: **Hello**, then **Everyone**.



Suppose Hello, 0 has already been sent. ACK 0 is on its way — but it is delayed.

Timer fires before ACK 0 arrives. What does the sender do?

> The **rdt 3.0 sender rule**: when the timer expires with no ACK heard, the sender assumes the worst and **retransmits the same packet**.
>
> So Hello, 0 goes out again, still numbered **0**.

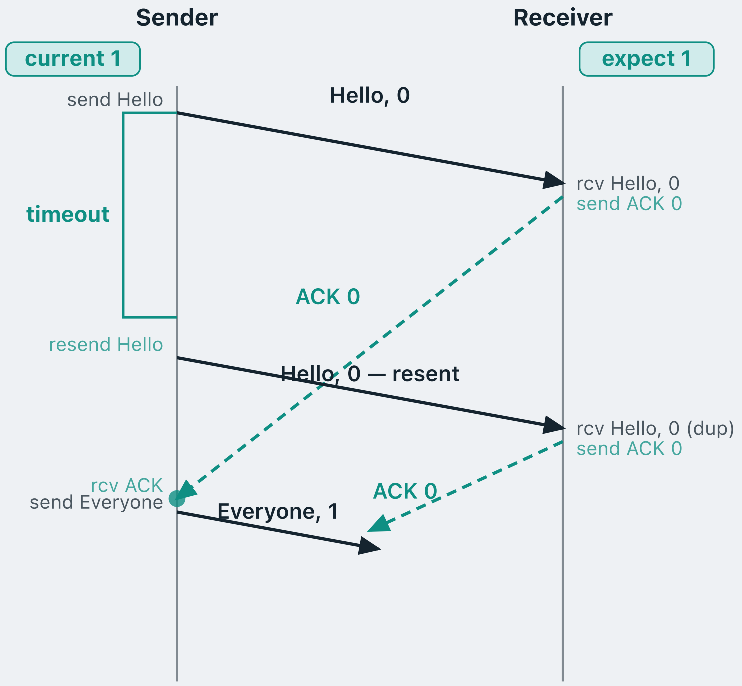
The receiver gets Hello, 0 a second time. What does it do?

> **rdt 2.2 receiver rule**: it expects **1**, this carries **0** — a duplicate. Discard it, and **ACK 0** goes back again.

The first ACK 0 arrives. What does the sender do?

A worked example

The sender wants to deliver two messages to the receiver: **Hello**, then **Everyone**.



Suppose Hello, 0 has already been sent. ACK 0 is on its way — but it is delayed.

Timer fires before ACK 0 arrives. What does the sender do?

> The **rdt 3.0 sender rule**: when the timer expires with no ACK heard, the sender assumes the worst and **retransmits the same packet**.
>
> So Hello, 0 goes out again, still numbered **0**.

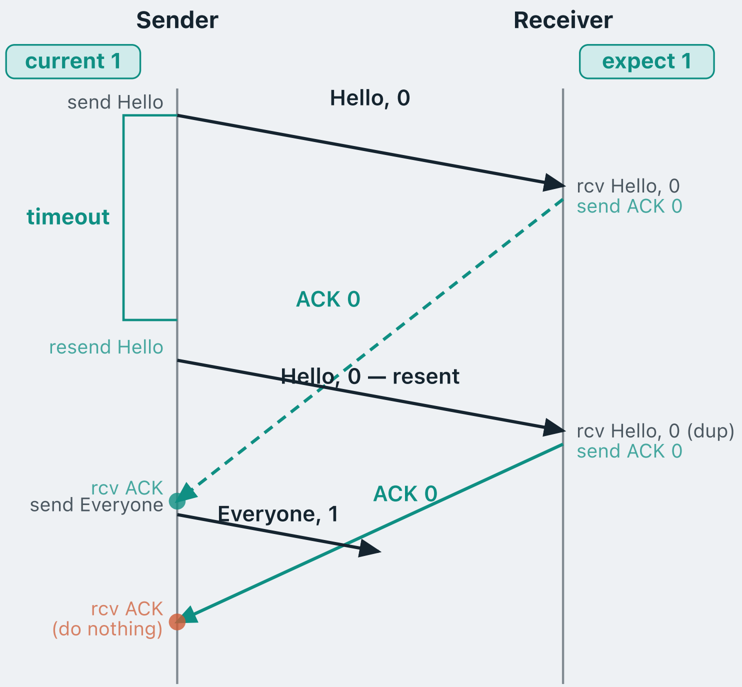
The receiver gets Hello, 0 a second time. What does it do?

> **rdt 2.2 receiver rule**: it expects **1**, this carries **0** — a duplicate. Discard it, and **ACK 0** goes back again.

The first ACK 0 arrives. What does the sender do?

> Move on — send the next packet, Everyone, 1 (flipped sequence number), and restart the timer.

What should a duplicate ACK do here?



The second ACK 0 now arrives. What should the sender do?

By the **rdt 2.2** rule, a second ACK 0 is a **duplicate ACK** — a NAK in disguise. So the sender should **retransmit Everyone, 1**.

But here that would be **wrong**.

This ACK 0 acknowledges the **previous** message, Hello, 0. It is a duplicate only because the first ACK 0 was **delayed**.

It was never meant to say that Everyone, 1 was corrupted, so it must **not** be treated as a NAK.

THE RETRANSMISSION RULE IN RDT 3.0

The sender retransmits **only when its timer expires**. Any ACK that is corrupt, or carries the wrong sequence number, is **ignored**.

That is the only difference from rdt 2.2 — the receiver is unchanged.

RECAP · STOP-AND-WAIT

What we covered

- **rdt 1.0** — over a perfectly reliable channel. No error handling needed: just send and forget.
- **rdt 2.0** — over a channel with bit errors. The receiver sends **ACK** or **NAK**; the sender retransmits on NAK.
- **rdt 2.1** — adds a **1-bit sequence number** to distinguish a resent packet from a new one.
- **rdt 2.2** — drops NAKs altogether. The receiver ACKs the **sequence number** it accepted, so a duplicate ACK is a NAK in disguise.
- **rdt 3.0** — over a lossy channel. A **timer** triggers retransmission if no expected ACK arrives in time.

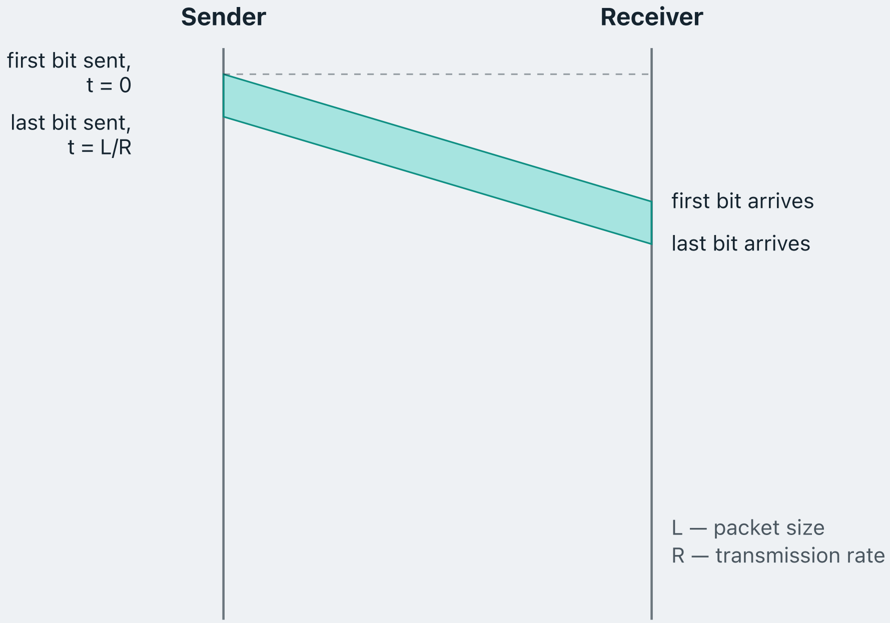
MODULE 4 · CONCEPT 3 OF 4

Go-back-N

Pipelined reliable data transfer

PERFORMANCE

The cost of stopping and waiting



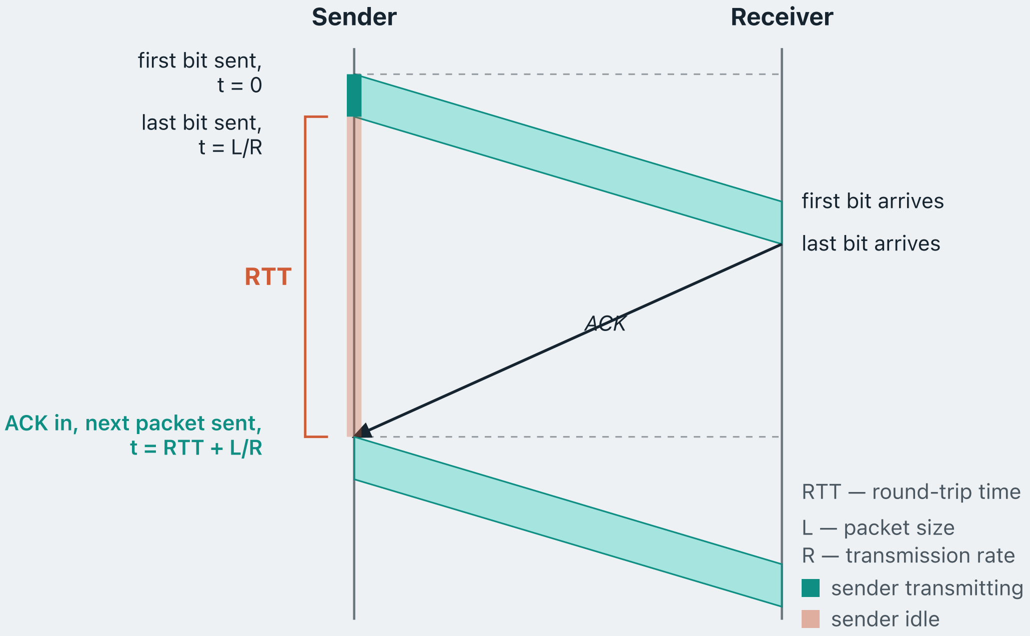
Every protocol we have built, rdt 2.0 through rdt 3.0, is **stop-and-wait**: send one packet, then sit idle until its ACK comes back.

That idle time is not free. Let us try to measure what it costs us.

Say every packet is **L** bits and the sender transmits at **R** bits per second.

Putting one packet onto the channel then takes **L/R** seconds.

How much of that time is the sender busy?



After that the sender must wait a full **RTT** for the ACK to come in.

Only then may it send again.

Therefore, sending one packet of length **L** takes the sender **RTT + L/R** seconds.

Of that, the sender is busy for only the first **L/R**, while it is transmitting. Every moment after that it is **idle**.

Fraction of the time the sender is busy:

`U = (L/R) ÷ (RTT + L/R)`

PERFORMANCE

How bad is it? A worked example

Take a 1 Gbps link, an RTT of 30 ms, and 1 KB packets.

$$L/R = 8000 \text{ bits} \div 10^9 \text{ bits/s} = \mathbf{8\ \mu s}$$

$$U = 0.008 \div (30 + 0.008) = \mathbf{0.00027}$$

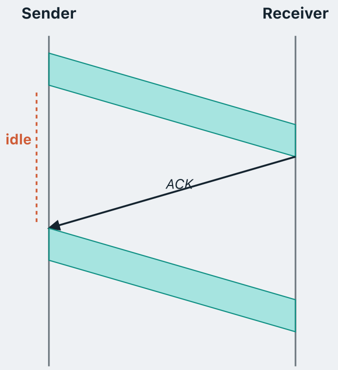
The sender is busy **0.027%** of the time. Put another way: a **gigabit** link delivers about **267 kbps** — roughly a thousandth of its capacity.

So how do we fix this?

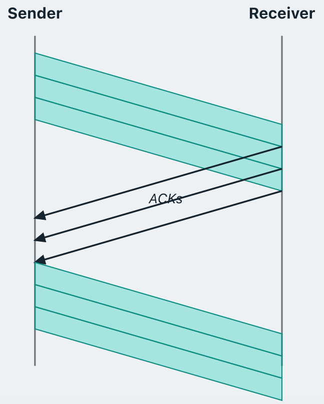
PIPELINING

Send several before you wait

Stop-and-wait — one at a time



Pipelined — several in flight



Recall the stop-and-wait utilisation: $U_{SW} = (L/R) \div (RTT + L/R)$

The channel has not changed, so the first ACK still returns at **RTT + L/R** — same denominator. But in that one window the sender has now sent **three** packets, so it was busy for **3(L/R)**:

$U_{pipe} = \mathbf{3}(L/R) \div (RTT + L/R)$

Three times the work in the same round trip — **3× more efficient**, as long as the three packets still fit inside it.

PIPELINING

What pipelining demands in return

Pipelining is not free. Two things have to grow.

A bigger sequence space

In stop-and-wait only one packet is ever in flight, so a single bit is enough: the receiver only has to tell “this one” from “the next one”.

With pipelining, however, we may have several packets in flight at once. If we pipeline three, the receiver has to tell the **first** from the **third** — one bit cannot do that.

Buffering

In a pipeline we have **multiple packets in flight** — already sent, but not yet acknowledged — and every one of them is liable to be retransmitted.

The sender therefore cannot just send and forget: it must remember every packet in flight, which is why it has to buffer **all** of them.

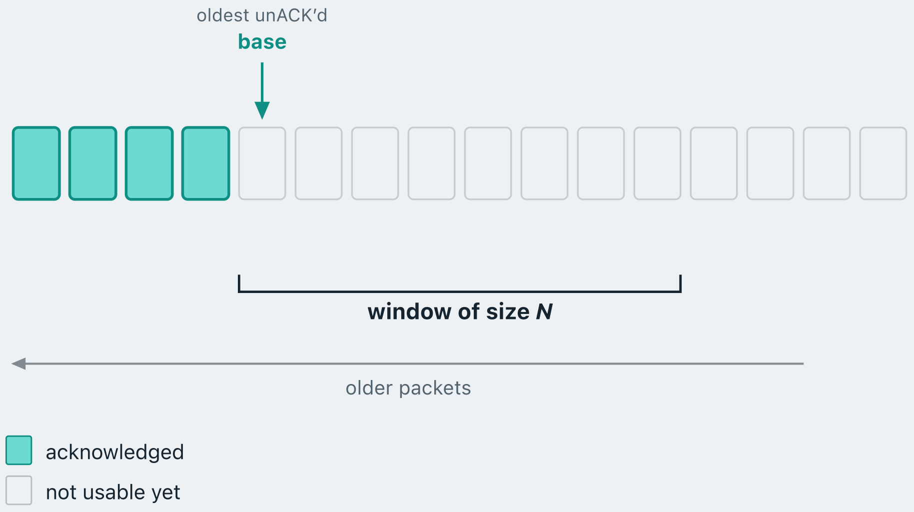
We will now learn about two pipelined protocols: **go-back-N** and **selective repeat**.

Go-back-N

- The sender keeps a **window** of up to *N* unacknowledged packets in flight at once.
- If a packet is lost or corrupted, the sender **goes back** to it and retransmits that packet together with every packet it has already sent after it — hence the name.

GO-BACK-N

The sliding window



The sender is fed a stream of packets from the application layer; each rectangle in the diagram is one packet.

Acknowledged — the sender has sent these packets, the receiver has received them intact, and an ACK has come back for each of them.

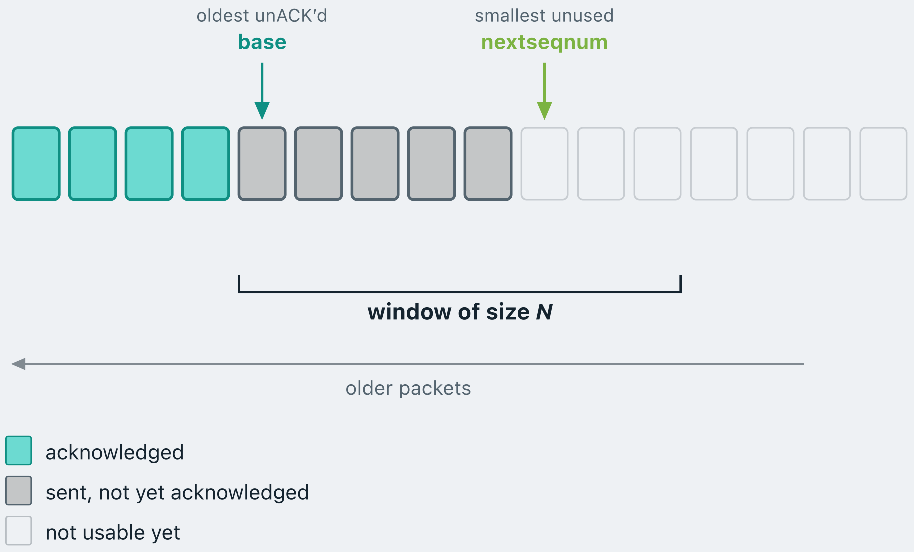
More data may arrive from the application, and in a pipelined protocol the sender would like to send it straight away. But how do we stop too many packets being in flight at once? This is what the **window** is for.

To place the window we first need a marker. **base** points to the oldest packet not yet acknowledged.

Window of size *N* — the window starts at **base** and spans the next *N* packets, so at most *N* unacknowledged packets are ever in flight.

GO-BACK-N

Filling the window

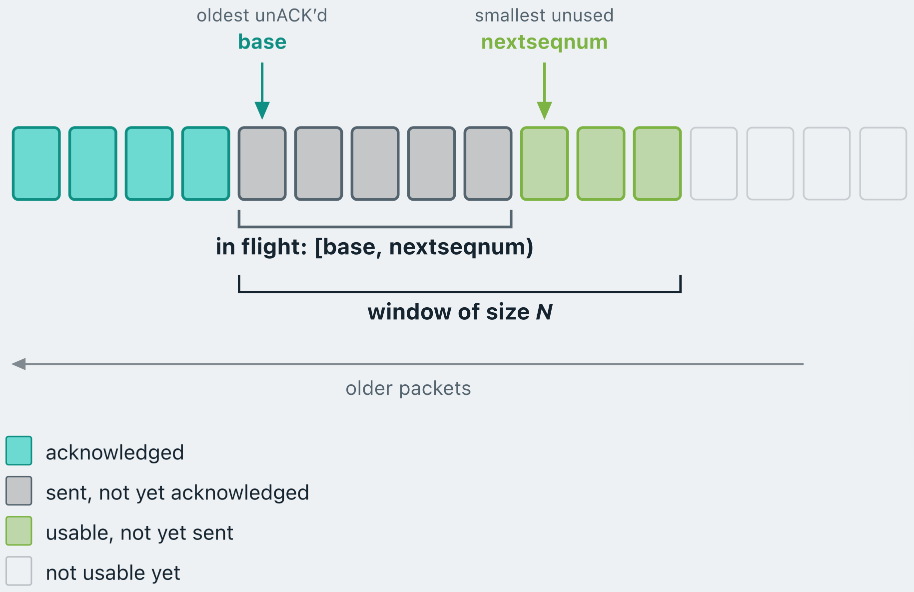


Sent, not yet ACK'd — suppose the application hands down five packets. The window is eight slots wide, so all five fit and go out at once. They are in transit and unacknowledged, so the sender keeps a copy of each in case it must retransmit.

Every packet sent uses up a sequence number, so after those five the next number to use sits here: **nextseqnum**.

GO-BACK-N

What is in flight?



The sender now tracks two numbers, **base** and **nextseqnum**. What does the range `[base, nextseqnum)` represent?

base marks the end of the acknowledged data and **nextseqnum** the end of the sent data. Everything between them is exactly what is **in flight**.

THE WINDOW CONSTRAINT

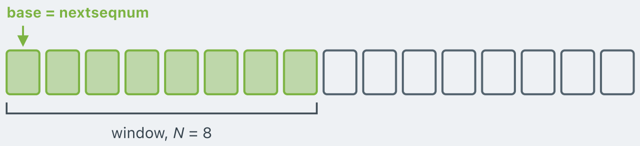
$$\texttt{nextseqnum} - \texttt{base} \le N$$

Once the range is full, the sender may not send again.

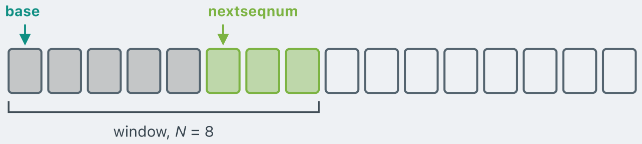
Usable, not yet sent — three window slots are still free, so the next three packets the application hands down can go straight out.

GO-BACK-N

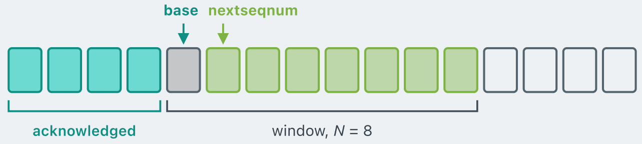
How does the sender window slide?



Nothing sent yet. The whole window is usable, and `base` and `nextseqnum` sit together.



Data arrives. Each time the application hands data to the sender, the sender transmits it and advances `nextseqnum`. Here five packets have gone out, so `nextseqnum` has advanced by five.



ACKs arrive. Suppose the receiver acknowledges the first four packets. Then `base` advances past everything acknowledged and the window slides right, opening up sequence numbers that were not usable before.

Cumulative acknowledgements

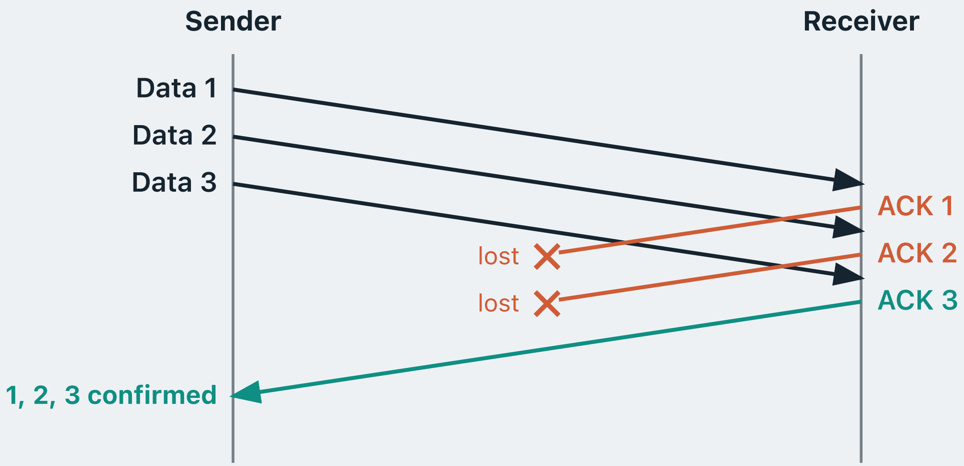
In Go-back-N the receiver does not acknowledge packets one at a time. It uses a **cumulative acknowledgement**.

> An **ACK *n*** means every packet up to and including *n* has arrived correctly — not just packet *n*, but everything before it too.

Suppose the sender sends **Data 1, 2 and 3** back to back.

ACK 1 and **ACK 2** are both lost in transit.

> **ACK 3 arrives** — and it alone tells the sender that 1, 2 and 3 all got through, even though ACK 1 and ACK 2 never made it.



The receiver’s rule in Go-back-N, informally

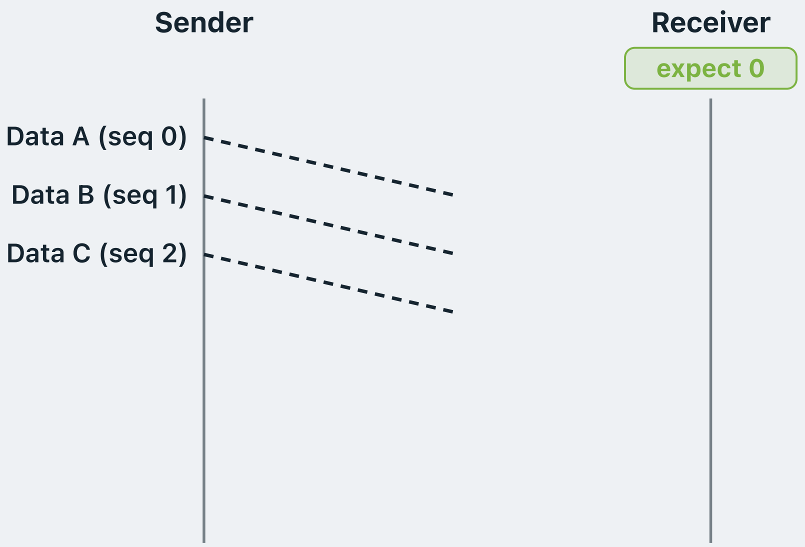
- Just as in rdt 3.0, the receiver keeps track of a single number, `expectedseqnum` — the sequence number of the **next in-order** packet it expects. (e.g. if it has received and delivered 4, 5 and 6 → `expectedseqnum = 7`)

- If the arriving packet is **intact** and is the **expected** one: **deliver** it and send an **ACK carrying its sequence number**.

- Otherwise — **corrupt**, or **not** the expected sequence number — **discard** the packet and **re-ACK** the last in-order sequence number, `expectedseqnum − 1`.

- In other words, the receiver accepts **in-order packets only** and discards everything else. It keeps **no buffer**.

The receiver in action

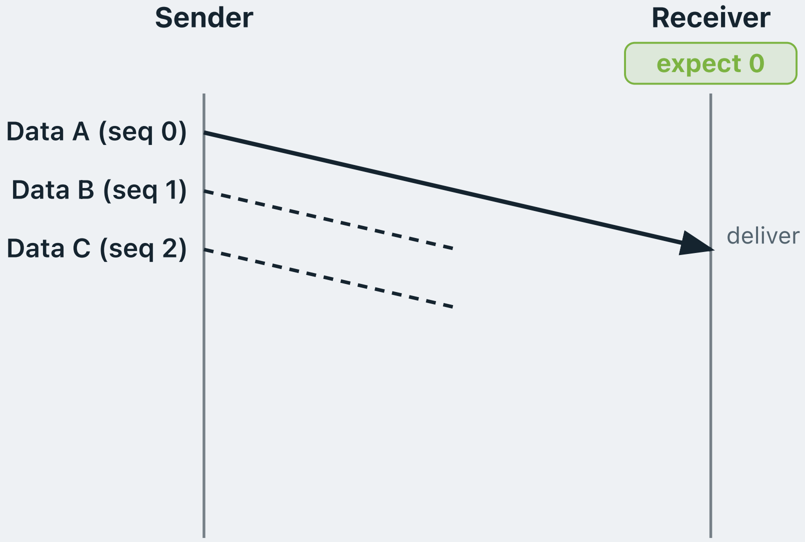


Suppose the sender is running Go-back-N with a window of $N = 5$, and the application hands it three packets: **Data A**, **B** and **C**.

Three is fewer than five, so all three fit in the window. The sender numbers them **0**, **1**, **2** and sends them together — all three are now in flight.

The receiver is waiting for the first in-order packet, so `expectedseqnum` is **0**.

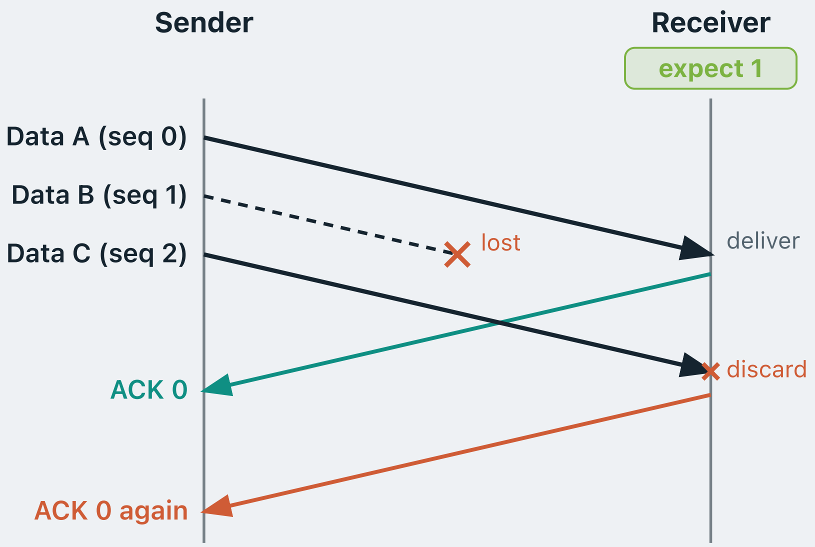
When a packet goes missing



Data A (seq 0) arrives. It is the expected one, so the receiver **delivers** it.

GO-BACK-N

When a packet goes missing



Data A (seq 0) arrives. It is the expected one, so the receiver **delivers** it.

It then sends **ACK 0** and advances `expectedseqnum` to **1**.

Data B (seq 1) is **lost** in transit. The receiver never sees it, so it is still expecting **1**.

Data C (seq 2) arrives, but seq 2 is not the 1 it expects. Out of order, so it is **discarded**.

The receiver then re-sends the ACK for the last in-order packet it delivered — here, **ACK 0**.

> That second **ACK 0** says: *I have still received everything up to sequence number 0, and nothing beyond it.*

GO-BACK-N

Receiver: the state machine

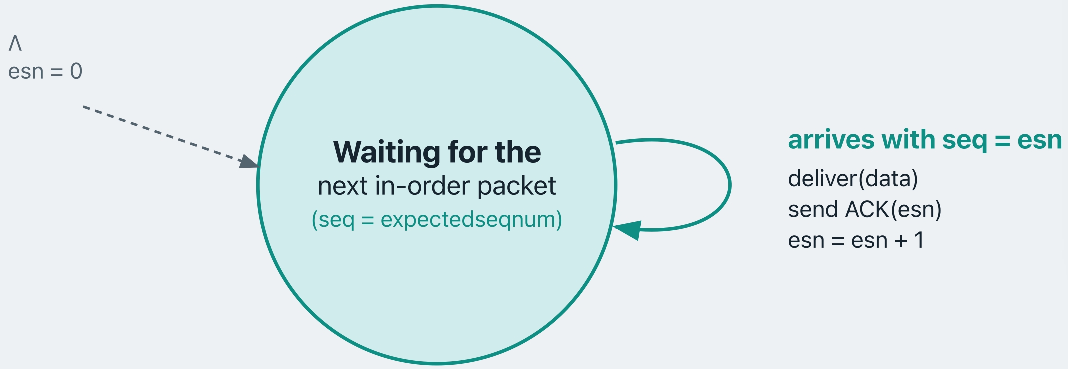
The state:

The receiver keeps a single number, `expectedseqnum` — the sequence number of the next in-order packet.

It sits in **one state**, waiting for that packet, and judges everything that arrives against that one number.

GO-BACK-N

Receiver: the state machine

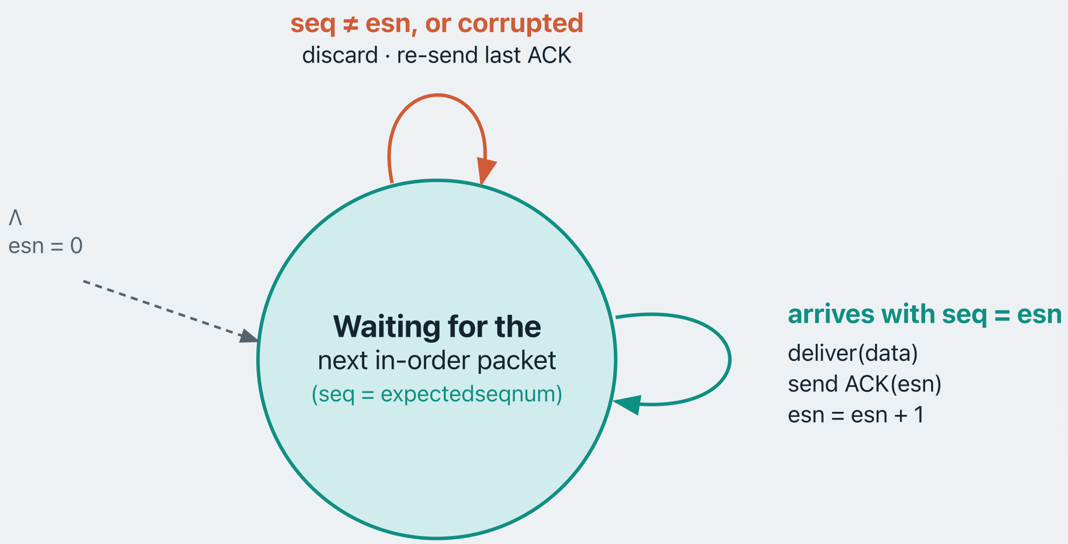


In-order packet:

```
if pkt.seq == expected_seq_num:
    deliver(pkt.data)
    send_ack(expected_seq_num)
    expected_seq_num += 1
```

GO-BACK-N

Receiver: the state machine

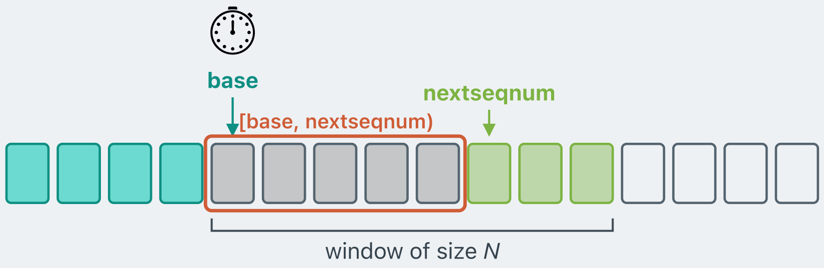


Out of order or corrupted:

```
else:
    discard(pkt)
    send_ack(expected_seq_num − 1)
```

GO-BACK-N

The retransmission rule in Go-back-N



This is why the sender keeps a **buffer** — a stored copy of every sent-but-unACK'd packet. Without it, go-back-N is impossible.

Just like in rdt 3.0, the channel can **lose** packets — Go-back-N still needs a **timer**.

The sender does **not** keep a timer per packet. It keeps **one timer for the whole window**, set on the **oldest unacknowledged** packet — the one `base` points to.

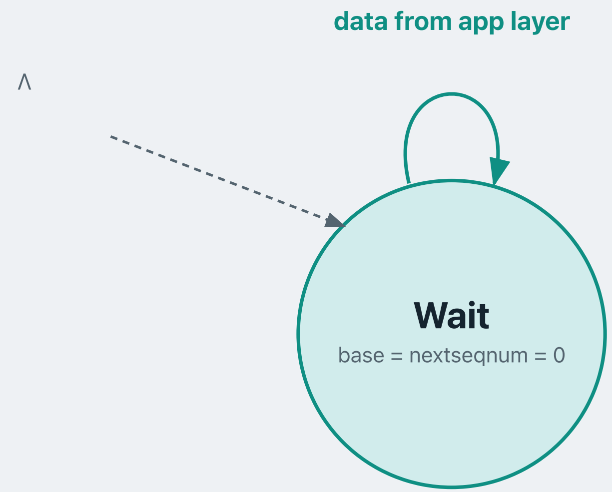
THE RETRANSMISSION RULE

When the timer expires, resend **every packet in flight** — that is, all of `[base, nextseqnum)`.

The reason: if `base` itself was lost, the receiver — following Go-back-N's rule — will discard every packet that arrived after the gap. The whole window must go again.

GO-BACK-N

Sender: the state machine



The sender sits in one **Wait** state, holding a window of at most *N* unacknowledged packets.

It tracks `base`, the oldest unACK'd packet, and `nextseqnum`, the next sequence number to assign.

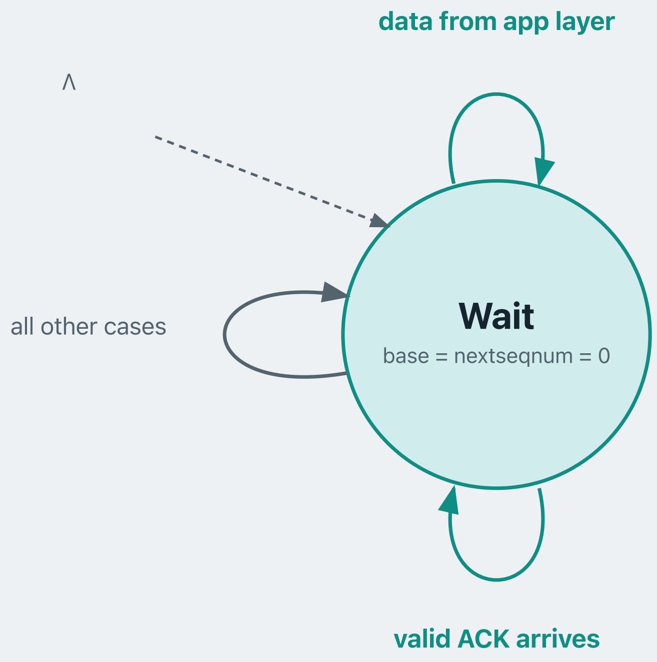
It also keeps `pkt_buffer` — a copy of every packet sent but not yet acknowledged.

Data from app layer:

```
if window not full (nextseqnum < base +
N):
    assign seq = nextseqnum
    buffer the packet and send it
    nextseqnum += 1
else:
    reject — window is full
```

GO-BACK-N

Sender: the state machine



The sender sits in one **Wait** state, holding a window of at most *N* unacknowledged packets.

It tracks `base`, the oldest unACK'd packet, and `nextseqnum`, the next sequence number to assign.

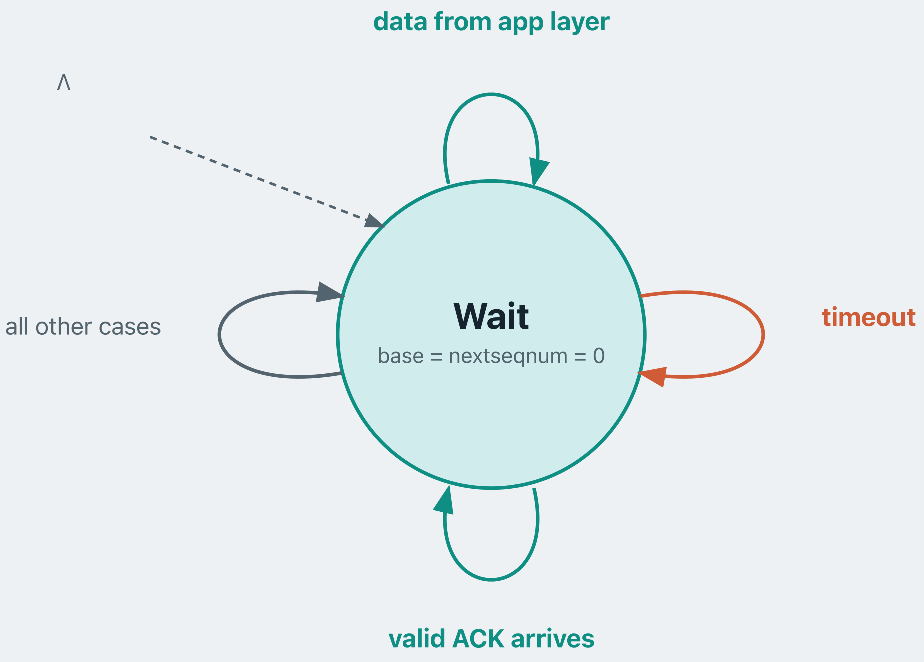
It also keeps `pkt_buffer` — a copy of every packet sent but not yet acknowledged.

An ACK arrives, carrying n:

```
if n ≥ base:
    base = n + 1 — slide the window
    restart timer for the new oldest
else:
    ignore it — do nothing
```

GO-BACK-N

Sender: the state machine



The sender sits in one **Wait** state, holding a window of at most *N* unacknowledged packets.

It tracks `base`, the oldest unACK'd packet, and `nextseqnum`, the next sequence number to assign.

It also keeps `pkt_buffer` — a copy of every packet sent but not yet acknowledged.

Timer fires:

```
restart timer
for every seq in [base, nextseqnum):
    re-send the buffered packet
```

RECAP · GO-BACK-N

What we covered

- **Sliding window** — the sender keeps a sliding window of at most *N* unacknowledged packets in flight.
- **Receiver rule** — the receiver sends a cumulative ACK and only accepts the next expected packet (`expectedseqnum`); it discards all out-of-order packets.
- **Timeout** — one timer covers the entire window; when it fires, the sender retransmits every packet currently in flight.

Selective repeat

Pipelined reliable data transfer

1 · Reliable data transfer | 2 · Stop-and-wait RDT | 3 · Go-back-N | **4 · Selective repeat**

GO–BACK–N

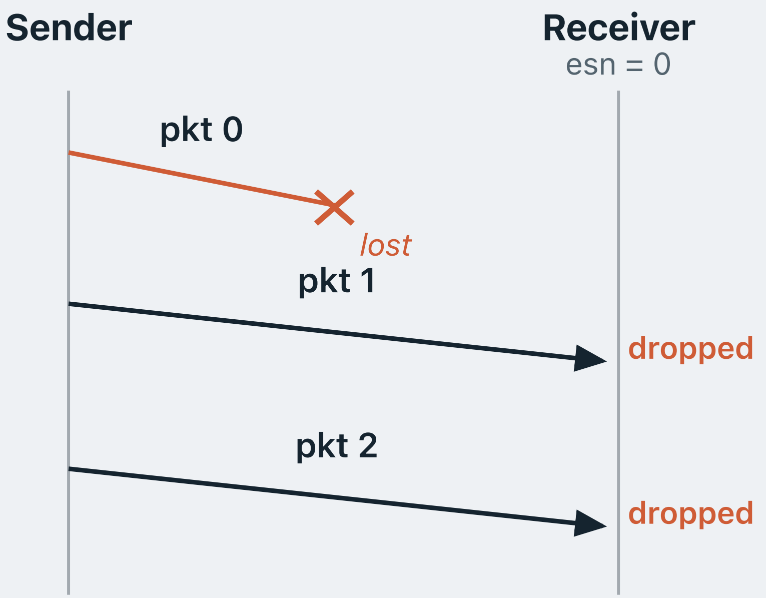
The cost of Go-back-*N*

Go-back-N wastes network capacity whenever packets arrive out of order.

Suppose pkt 0 is lost in transit. The receiver's `expectedseqnum` stays at 0.

> pkt 1 and pkt 2 arrive correctly — but because `expectedseqnum` = 0, both are **discarded**.

Can we do better?



SELECTIVE REPEAT

Introducing Selective Repeat

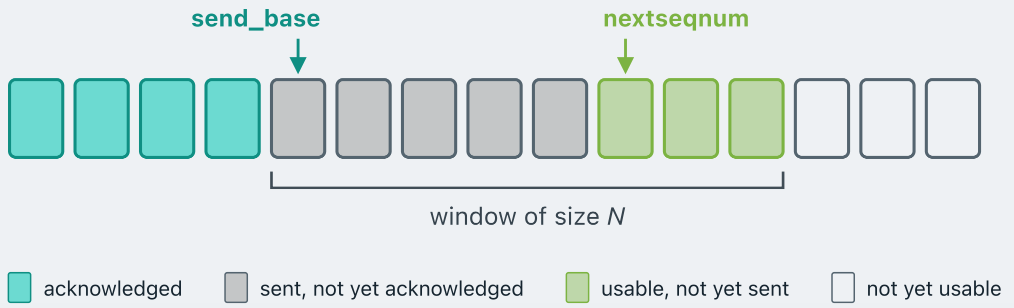
The receiver keeps a buffer

Out-of-order packets are **buffered**, not discarded.

The sender resends only the packets that are truly lost

SELECTIVE REPEAT

The sender's window in Selective Repeat

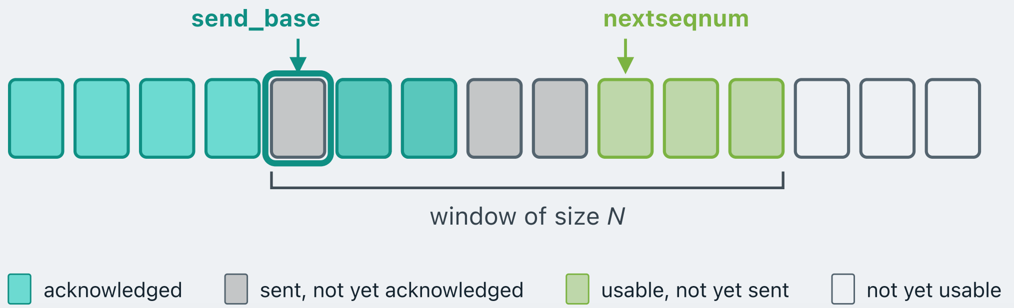


`send_base` always points to the **oldest unacknowledged** packet. The window always starts from `send_base`.

Selective Repeat uses **individual** ACKs, not cumulative ones — each ACK names exactly one packet.

SELECTIVE REPEAT

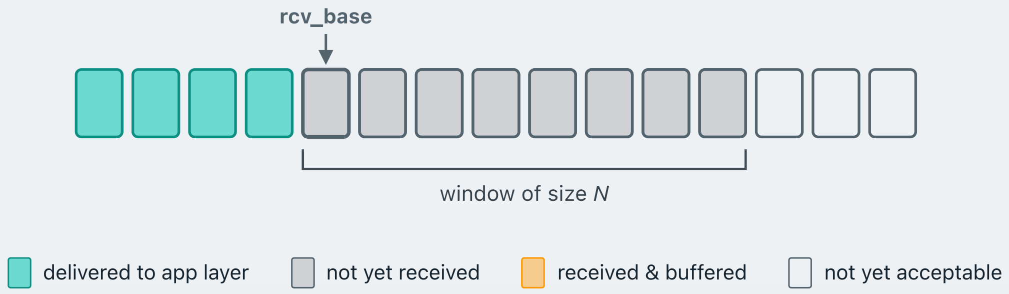
Holes in the sender’s window



Because ACKs are individual, the sender’s window can have **holes**. Here, for example, the two packets after `send_base` are acknowledged while `send_base` itself is not, leaving a hole at the front of the window.

In Go-back-N this is impossible: a cumulative ACK acknowledges **every** packet before it, so there can never be a hole in the sender’s window.

The receiver's window in Selective Repeat



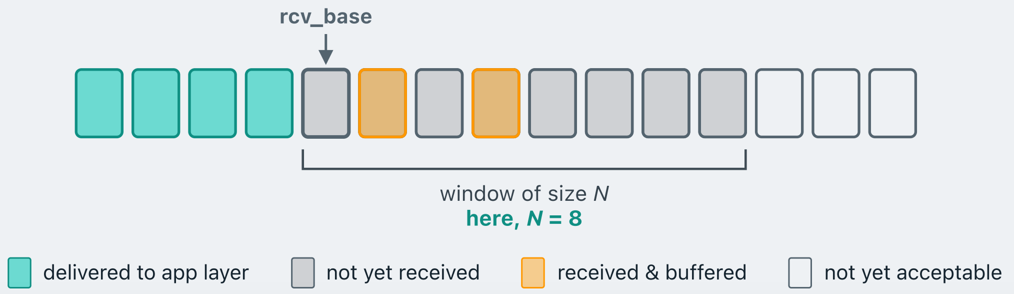
The receiver **buffers** out-of-order packets rather than discarding them. But its buffer is finite, so it can only accept a finite number of them.

That is why the receiver has a **window**: it fixes the range of out-of-order packets it is willing to buffer.

The window starts at `rcv_base`, which points to the **first packet not yet delivered** — everything below it has arrived in order and gone to the application.

SELECTIVE REPEAT

How much will the receiver buffer?



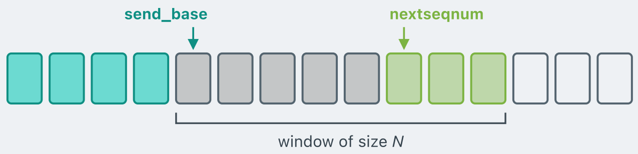
Here the window is of size **8**, so the receiver is willing to buffer the next **8** packets after `rcv_base` if they arrive out of order — instead of discarding them, as Go-back-N would.

In this diagram, two packets have arrived inside the window and are being **buffered**.

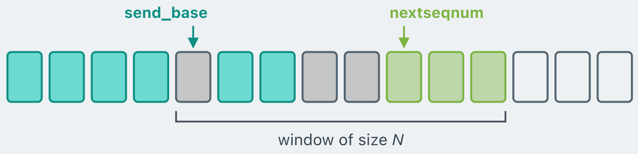
The receiver's window is normally the **same size as the sender's**, so it can buffer every packet the sender may have in flight.

SELECTIVE REPEAT

How the sender’s window slides forward

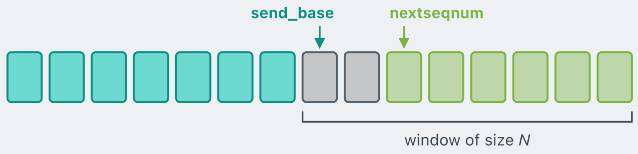


Before. Five packets are outstanding, none acknowledged yet.



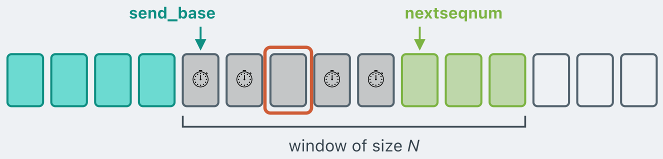
Two ACKs arrive. They are individual ACKs for two packets **after** `send_base`, so those two slots are marked acknowledged.

- Notice those two are no longer in flight, and they free up space — can we slide the sender’s window forward by 2?
- **No.** By definition the window always starts at `send_base`, the **oldest unacknowledged** packet. Here `send_base` has not changed, so the window cannot slide.
- **Only when send_base itself is acknowledged does the window advance.**



Suppose the packet at `send_base` now arrives. `send_base` advances past it **and past the two slots already acknowledged behind it**, landing on the next packet still outstanding — so the window slides forward by **3** in a single step.

Timeout: resend just the one



Each outstanding packet has **its own timer**.

When any timer expires, the sender resends **that packet only** and restarts **that** timer. Everything else in the window is untouched.

This is in contrast with Go-back-N, where one timer covered the whole window and one expiry resent all packets in flight.

This is where Selective Repeat wins: only the packets that were **truly lost** go out again, so far fewer retransmissions than Go-back-N.

SELECTIVE REPEAT

The sender’s rules

`data received from above`

If the next available sequence number falls **inside the window**, send the packet.

`timeout`

Every packet has its own timer. Resend **that one packet** and restart its timer.

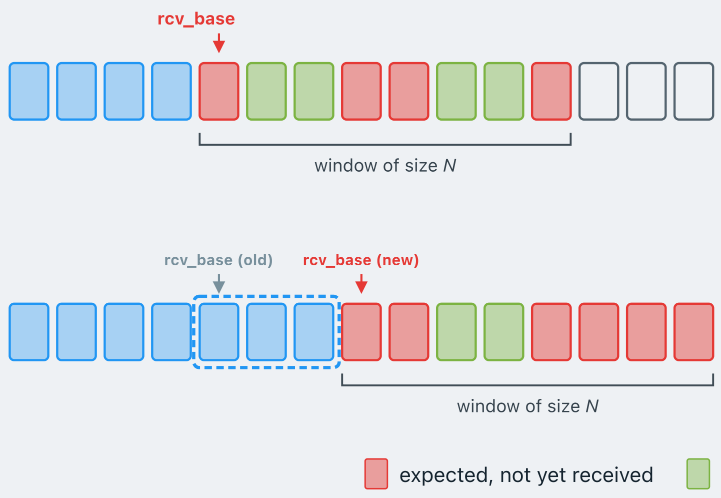
`ACK received, seq# in [send_base, send_base+N-1]`

Mark that packet as received.

If its sequence number **equals `send_base`**, advance the base to the next unacknowledged number.

SELECTIVE REPEAT

How the receiver’s window slides forward



A packet arrives within the window, but it is not at `rcv_base`.

Buffer it, and send an **individual ACK** for it.

`rcv_base` does not move: by definition it points to the **first packet not yet delivered**, and that packet is still missing. So the receiver’s window stays in place.

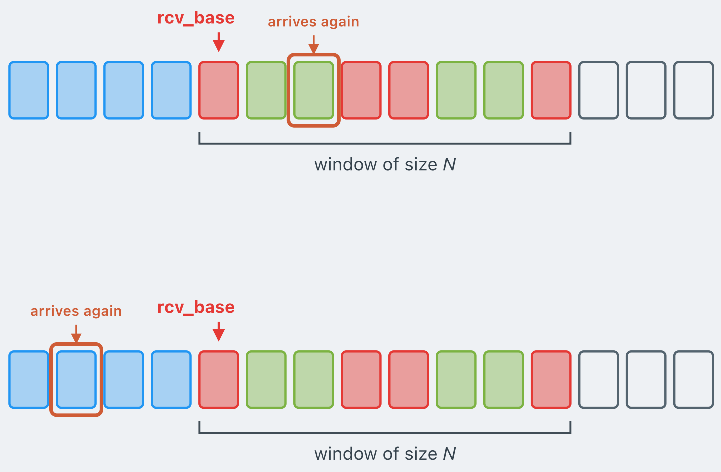
The packet at `rcv_base` arrives.

Deliver it to the application, then deliver every consecutively buffered packet above it.

Advance `rcv_base` past all of them — the window jumps forward in one go.

SELECTIVE REPEAT

Corner cases for the receiver



expected, not yet received · received and buffered · delivered to application

A packet that is already buffered arrives again.

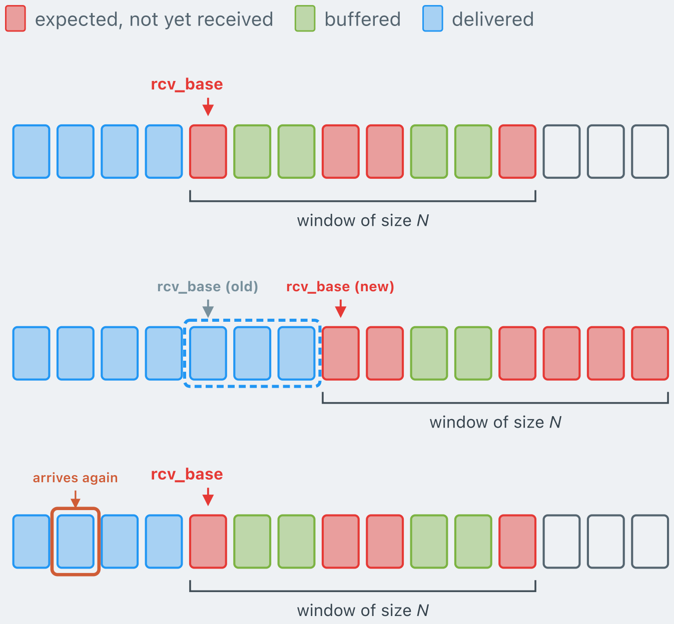
- Why would the sender send it a second time?
- The first individual ACK may have been **lost or corrupted** in transit, so the sender **timed out** on this packet and retransmitted it.
- What should the receiver do?
- **Ignore** the duplicate. Re-send the **lost individual ACK**.

A packet from *below* the receiver's window arrives again.

- The individual ACK for this old packet clearly did not get through — the sender timed out and retransmitted.
- **Re-send the lost ACK**.

SELECTIVE REPEAT

The receiver’s rules



When a new packet arrives, look at its seq#, then:

seq# in [`rcv_base, rcv_base+N−1`]

- Send an **individual ACK**.
- Out of order and not yet buffered? **Buffer it.**

Equals `rcv_base`?

- Deliver it, together with any consecutively buffered packets above it. **Advance the window** past all of them.

Otherwise — below the window

- The packet was already delivered. **Re-send the ACK** — the first one must have been lost.

SELECTIVE REPEAT

A dilemma when window size exceeds the seq# space

In practice sequence numbers come from a **finite** space, so they **wrap around**.

Say they are modulo **3**. Then they run ..., 0, 1, 2, 0, 1, 2, ...

What problem could that cause? Take a window of size **3**, and consider two different histories.

The receiver cannot tell which

The **sender** knows which history it is in — it knows whether it is retransmitting or sending genuinely new data.

The **receiver** sees only a packet with **seq# = 0**. The two histories look identical to it — just as in **rdt 2.0**, it cannot tell a retransmission from genuinely new data. The ambiguity is back because the sequence space is **not large enough** to keep the two apart.

SELECTIVE REPEAT

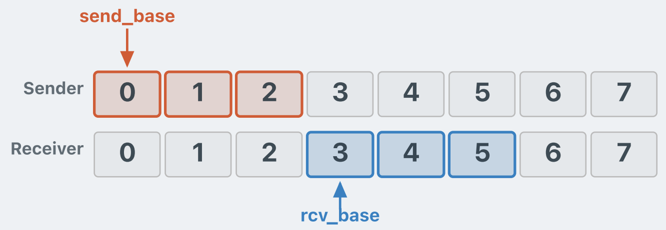
Why the two windows drift apart

The deeper reason is that the sender’s window and the receiver’s window can **drift far enough apart to overlap after wrap-around**.

Suppose the window size is **3**, and number the packets straight through, 0, 1, 2, 3, ... At first the two windows line up — `send_base` and `rcv_base` point at the same packet.

Why the two windows drift apart

The deeper reason is that the sender’s window and the receiver’s window can **drift far enough apart to overlap after wrap-around**.



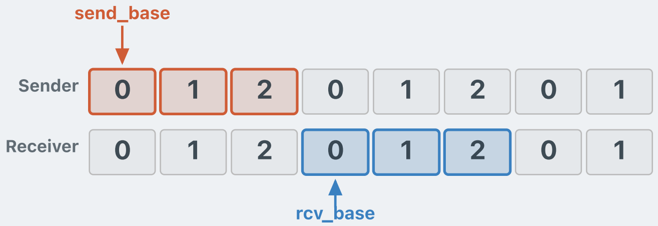
Suppose the window size is **3**, and number the packets straight through, 0, 1, 2, 3, ... At first the two windows line up — `send_base` and `rcv_base` point at the same packet.

Now packets 0, 1 and 2 all arrive, but the **ACK for packet 0 is lost**. The receiver delivers all three and moves `rcv_base` to 3, while the sender is still waiting on 0 — the windows have **drifted**.

SELECTIVE REPEAT

Why the two windows drift apart

The deeper reason is that the sender's window and the receiver's window can **drift far enough apart to overlap after wrap-around**.



Suppose the window size is **3**, and number the packets straight through, 0, 1, 2, 3, … At first the two windows line up — `send_base` and `rcv_base` point at the same packet.

Now packets 0, 1 and 2 all arrive, but the **ACK for packet 0 is lost**. The receiver delivers all three and moves `rcv_base` to 3, while the sender is still waiting on 0 — the windows have **drifted**.

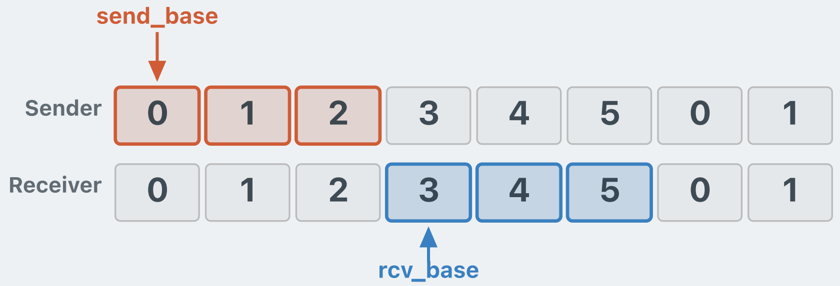
This would be a problem if the sequence number space is **small** — e.g. if it is modulo **3**, so the labels run 0, 1, 2, 0, 1, 2, …

Now notice that `send_base` and `rcv_base` both read **0**, even though they are three packets apart. The wrap-around has hidden the gap.

So if the sender retransmits its first packet after a lost ACK, the receiver sees a **0** that falls inside its own window, and takes it for new data.

SELECTIVE REPEAT

Window size vs. seq-number space



So how do we stop the two windows colliding?

THE RULE window size ≤ ½ × seq# space

With a window of **3**, the sequence space must run at least **0–5**. Then `send_base` reads **0** and `rcv_base` reads **3** — the two can no longer be confused.

Think about it: why is this exactly the right requirement?

RECAP

Pipelined protocols — what we covered

Stop-and-wait sends one packet at a time, which leaves the link idle for almost the whole round trip. **Pipelining** keeps several packets in flight, capped by a **window**.

- **Go-back-N** — cumulative ACKs, one timer for the whole window, and a receiver that discards anything out of order. A single loss resends everything in flight.
- **Selective repeat** — individual ACKs, a timer per packet, and a receiver that buffers out-of-order packets. Only what was actually lost is resent.

Selective repeat costs buffering at both ends, and it needs `window size ≤ ½ × seq# space` so the two windows can never be confused after wrap-around.

References

James F. Kurose & Keith W. Ross, *Computer Networking: a Top-Down Approach Featuring the Internet*, Addison Wesley, 3rd edition, 2005.

Chapter 3 — The Transport Layer (§3.4)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

Chapter 7 — Data Link Control Protocols

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

Chapter 3 — The Data Link Layer (§3.3, §3.4)

Acknowledgements

These lecture notes are developed based on slides from the following sources:

- **Dr Barry Wu**'s slides for COSC264, University of Canterbury, 2024–25
- **Dr Mengmeng Ge**'s lecture notes for COSC264, University of Canterbury, 2024
- **Assoc Prof Dan Kim**'s lecture notes for COSC264, University of Canterbury, 2017
- **Prof Aleksandar Kuzmanovic**'s lecture notes for CS340, Northwestern University, 2005, https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm

Lineage & colophon

These lecture notes modernise COSC264 material developed by **Dr Barry Wu** (2024–25), building on notes by **Dr Mengmeng Ge** (2024) and **Assoc Prof Dan Kim** (2017), University of Canterbury, and lecture notes by **Prof Aleksandar Kuzmanovic** (CS340, Northwestern University, 2005).